

Document Structure



Prominent Point Phase Curvature Autofocus Tests on Cubic Spline Path

Prepared by:
Ramakrishna Sen
Signal Processing R&D Team

Table of Contents

Table of Contents.....	2
Introduction.....	2
Theory.....	3
Block diagram.....	3
Testcases.....	7
High-frequency Low Amplitude sinusoidal noise.....	8
Low-frequency High-Amplitude sinusoidal noise.....	10
Medium frequency medium amplitude sinusoidal noise.....	12
High frequency + Low Frequency Sin noise (reproc).....	14
Gaussian noise without filtering.....	15
Lowpass filtering through Gaussian filter (noise smoothening).....	15
Unfiltered Gaussian noise.....	16
Filtered Gaussian noise (lowpass sigma = 10).....	18
Sensor-1 IdeaForge Platform Test.....	20
Overall Comparison.....	20
Corner Reflector Level comparison.....	21
Sensor-1 Scandron platform test.....	26
-----	30
Conclusion.....	33

Introduction

This document outlines the implementation and validation of a Prominent Point Phase Curvature Autofocus (PP-PCA) algorithm for SAR imaging in drone-borne systems. The algorithm intends to correct the residual motion errors, even after initial Inertial Navigation System (INS) based compensation (called mo-co). The PP-PCA algorithm extracts this phase history from the range history and then . The methodology involves an iterative process of initial image formation, target extraction, phase curvature estimation via a Maximum Likelihood-derived estimator and other filtering steps. The detailed description of the algorithm has been provided in this document. The validation is performed using simulated datasets with known motion errors, demonstrating the algorithm's ability to accurately recover the injected phase distortions and improve image focus. Finally this was sealed off by successfully testing this out on Sensor-1 on both platforms (IdeaForge and Scandron)

Theory

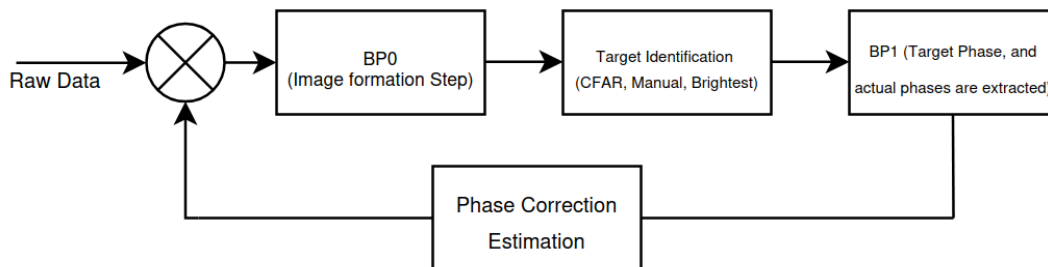
Prominent Point Phase Curvature Autofocus (PP-PCA) is a novel data-driven autofocus technique used in Synthetic Aperture Radar (SAR) to correct for unknown motion errors that would otherwise blur the image. Instead of relying on external motion sensors, PPPCA uses the radar data itself.

It works by identifying one or more strong, stable "prominent points" in the scene. These points are used as references to estimate and remove phase errors from the dataset. A single point can correct for translational motion, while using multiple points allows for the correction of more complex rotational motion and the accurate scaling of the final image. This idea follows from Carrara et al. 268 which uses the RCS from the prominent targets to correct for the whole image. However the drawback of such an approach is that this process is manual and often adds a burden to the processing time. Instead of adopting the conventional Prominent Point approach, we developed a fusion of the idea with the phase curvature autofocus algorithm to come up with an automatic focusing algorithm which does not require a manual involvement for target selection.

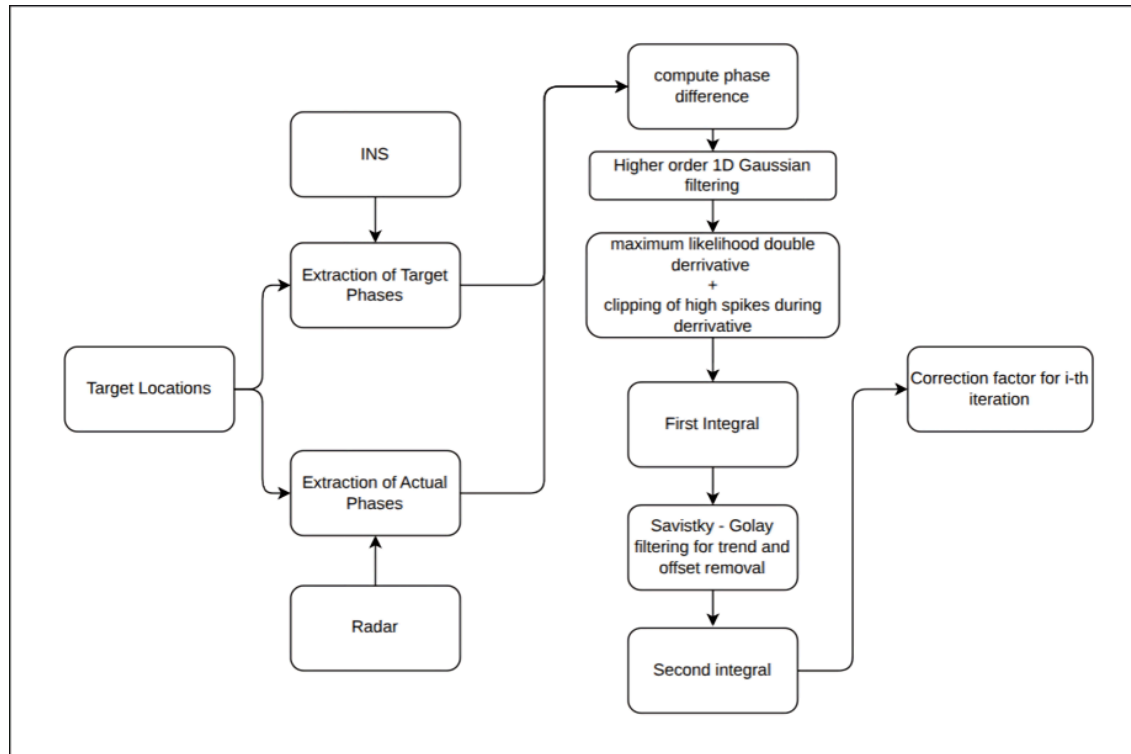
The Phase Curvature Autofocus (PCA) is a non-parametric autofocus technique which estimates motion errors by measuring the second derivative of the signal's phase—the "phase curvature"—from prominent targets. By averaging these curvature measurements, it can accurately correct for quadratic and other higher-order phase errors that cause image defocusing. An extension of this idea is discussed in the text by Saeedi.

Block diagram

Overall autofocus backprojection process



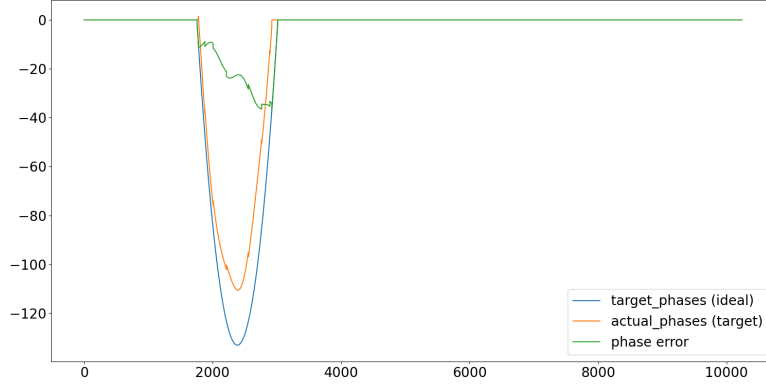
Elaborated BP1 Block



The proposed algorithm uses the idea from the above mentioned ideas and incorporates in the stripmap mode SAR. The algorithm “compares” the phases of prominent targets from the INS input and from the raw data. Since the motion-compensation algorithm takes into uses the INS data, it is limited by the measurement accuracy of the INS. Our proposed algorithm attempts to correct phase errors below the bounds of the INS accuracy (nearly 2cm path deviations).

The phase curvature of both the sources must be similar, and since the moco utilizes the INS data, we try to correct the INS phase based on the sensor data.

$$\Delta\phi = \frac{-4\pi R}{\lambda}$$



Once we have the reference phases from the range, we compute the phase difference

$$\Delta\phi_k = \phi_{k,\text{actual}}(n) - \phi_{k,\text{INS}}(n)$$

$$\Delta\phi_k = \arg(S_k(n)) - \frac{4\pi}{\lambda} R_{k,\text{INS}}(n)$$

Where k denotes the target index and S is the complex raw SAR data obtained from the radar. All of this gets accumulated into a single correction matrix ϕ_{err} .

$$\phi_{err} = \sum_{k=1}^K \Delta\phi_k(n)$$

Since we intend to remove the quadratic components present in the raw data, our goal is to bring the curvature of the INS phase as close as possible to the raw data phase, whereby the motion compensation would be virtually perfect. Therefore we want our difference phase signal to be as free of higher order terms as possible. To get that our idea is to regenerate that signal by a double derivative and a double integral step. In theory, this should be enough to remove any trends, however due to the nature of the operation, we require two very crucial steps involved in the process. A smoothening filter before the derivative step; and a detrend and reconstruction filter before the second integral. This is because the derivative is a high-pass operation. It amplifies the inherent high frequency presence in the difference phase. Hence a smoothening operation is necessary. Similarly the integral operation adds offsets to the resulting signal. A Savitsky-Golay filter tries to fit a polynomial into the signal which removes the trends which were added during the first integral operation. One can even add another block of the same filter after the second, but experimentally it was found to be sufficient without it.

Therefore after we have our phase difference, we pass it through a smoothening filter, which is simply the convolution of the signal with a Gaussian Kernel $h(m)$, where

$$h(m) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{m^2}{2\sigma^2}}$$

Hence after the convolution step, we arrive a

$$\phi(n) = \phi_{\text{err}}(n) * h(n) = \sum_{m=-\infty}^{\infty} \phi_{\text{err}}(n - m) \cdot h(m)$$

A sequence of operations now follows. The second derivative is approximated using a second-order central finite difference formula. This method provides a robust estimate of the curvature at each sample point. The resulting signal, representing the phase curvature, is denoted and is calculated as follows

$$\ddot{\phi}(n) \approx \frac{\phi(n+1) - 2\phi(n) + \phi(n-1)}{(\Delta n)^2}$$

For usual sampling, the interval is 1. So the expression becomes

$$\ddot{\phi}(n) = \phi(n+1) - 2\phi(n) + \phi(n-1)$$

$$\dot{\phi}(n) = \sum_{i=0}^n \ddot{\phi}(i)$$

The Savitzky-Golay filter is employed to remove the trends without distorting the desired underlying signal. It operates by fitting a low-order polynomial to the data within a moving window using the method of linear least squares. For each point n , a polynomial $P(m)$ of a given order d is fitted to the signal samples in a window of size $2W+1$ centered at n .

$$P(m) = \sum_{j=0}^d c_j m^j, \quad m \in [-W, W]$$

We remove the predicted trend from the signal which gives

$$\dot{\phi}_{\text{clean}}(n) = \dot{\phi}_{\text{est}}(n) - P(0)$$

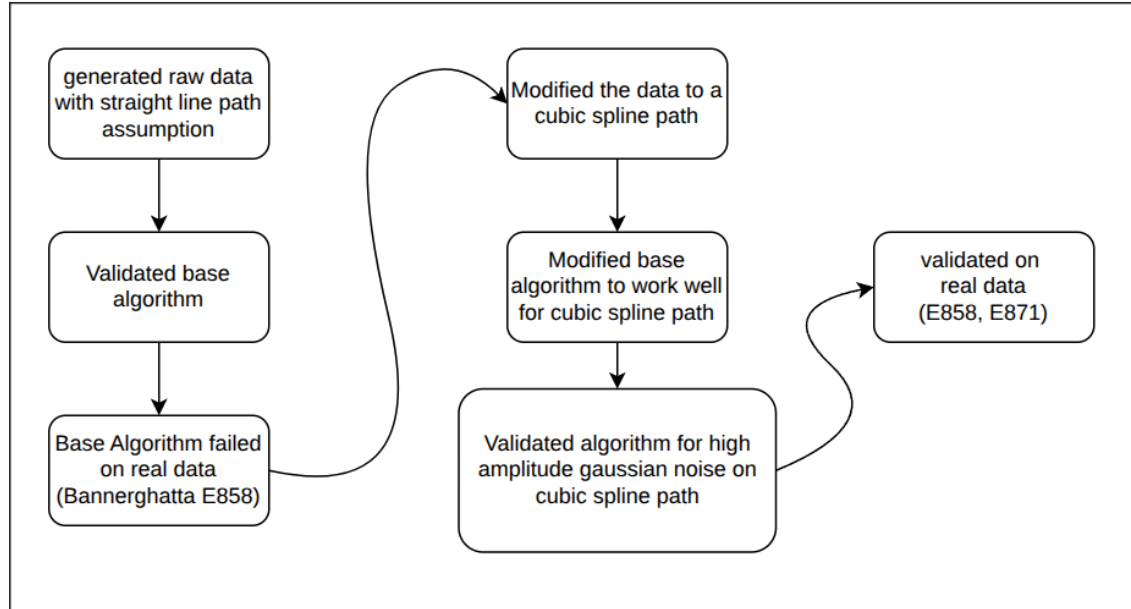
Finishing up with the summation step, we get

$$\phi_{\text{correction}}(n) = \sum_{i=0}^n \dot{\phi}_{\text{clean}}(i)$$

This is the correction we would be applying to all range bins.

Testcases

The testing methodology involved drawing conclusions from the straight line assumption PP-PCA from [this](#) document. The block diagram briefly describes the testing process followed during the development of this algorithm.



The number of errors which were tested out on the cubic spline path are discussed under this section. Please note that the term **high-amplitude** in the tests conducted indicate that the deviation in the flight path is "noticeable" while simulating the same. Usually for the tests conducted, higher amplitude refers to the values closer to the INS RTK accuracy range (2cm).

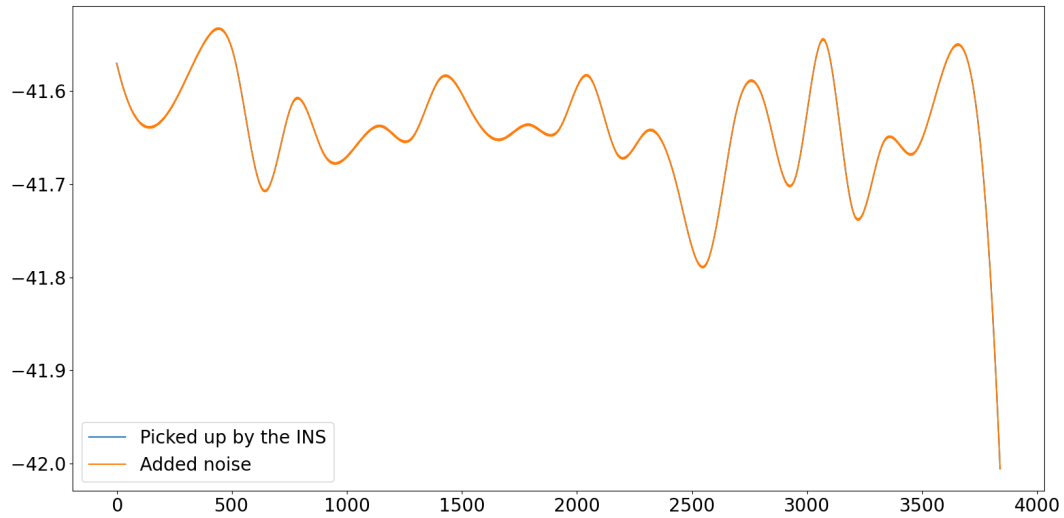
The format of the tests include

- Flight path deviation (for simulated data)
- Number of iterations processed
- Base-Image and Final-Image qualitative comparison
- Azimuth PSLR comparison (BI-FI comparison)
- Image sharpness and Entropy Value (BI-FI comparison)

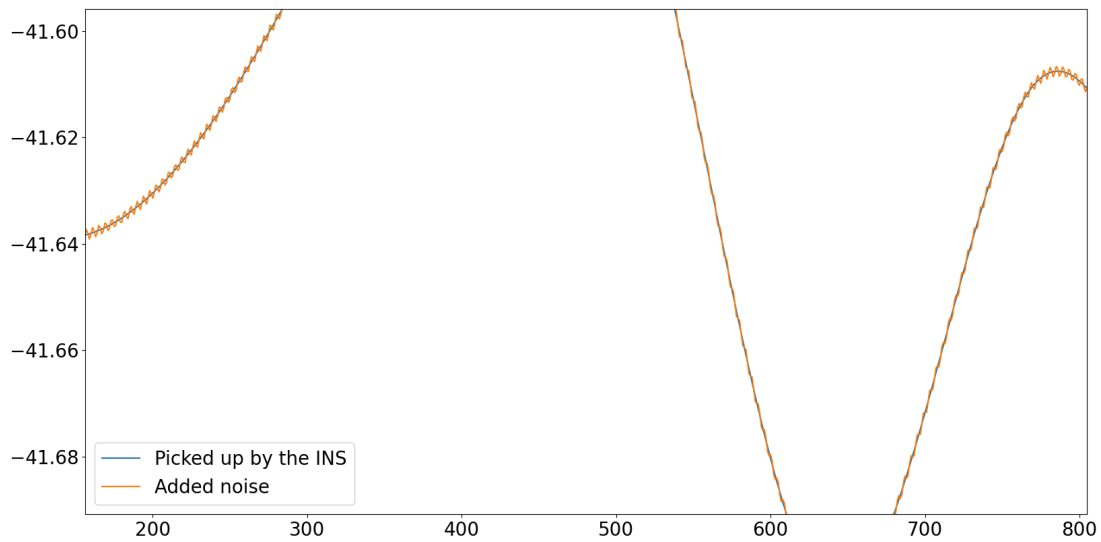
High-frequency Low Amplitude sinusoidal noise

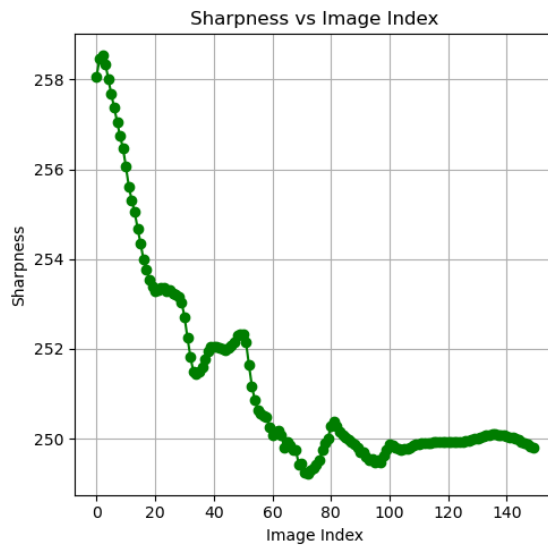
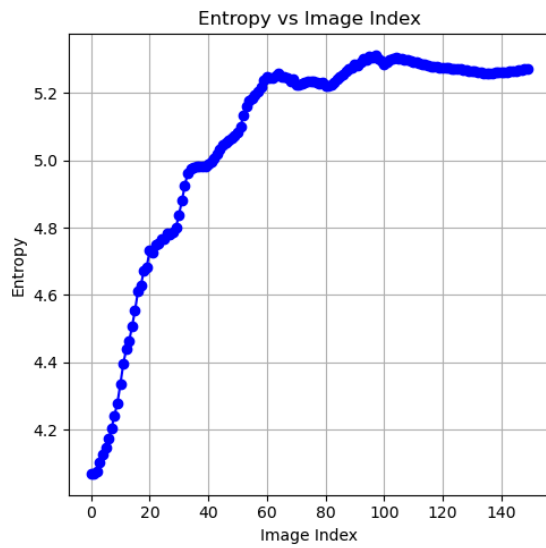
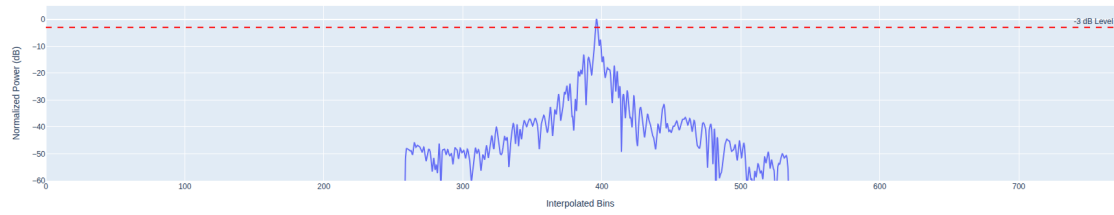
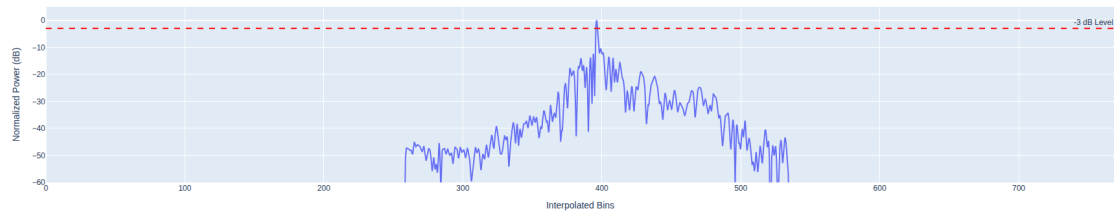
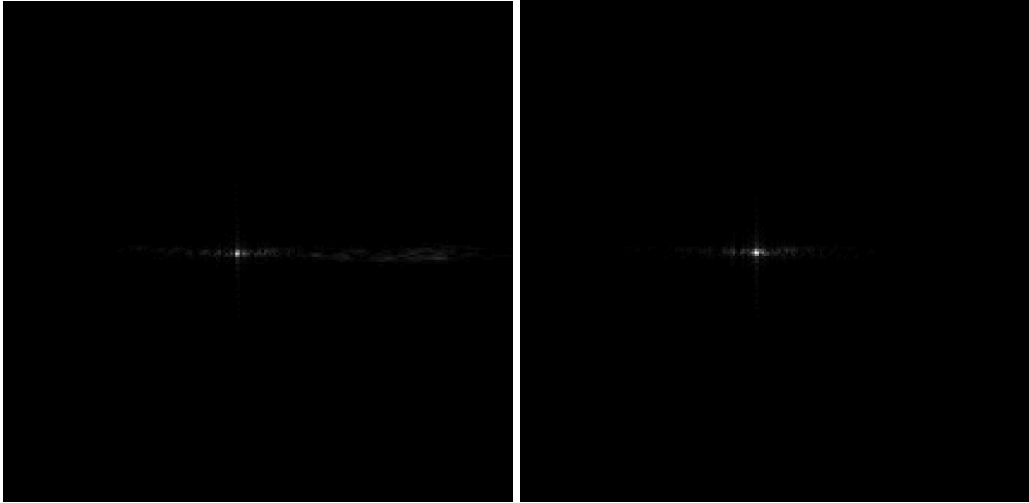
```
noise = 0.001 * np.sin(2*np.pi*50*time)
```

1mm amplitude



(zoomed in version, just for clarity)

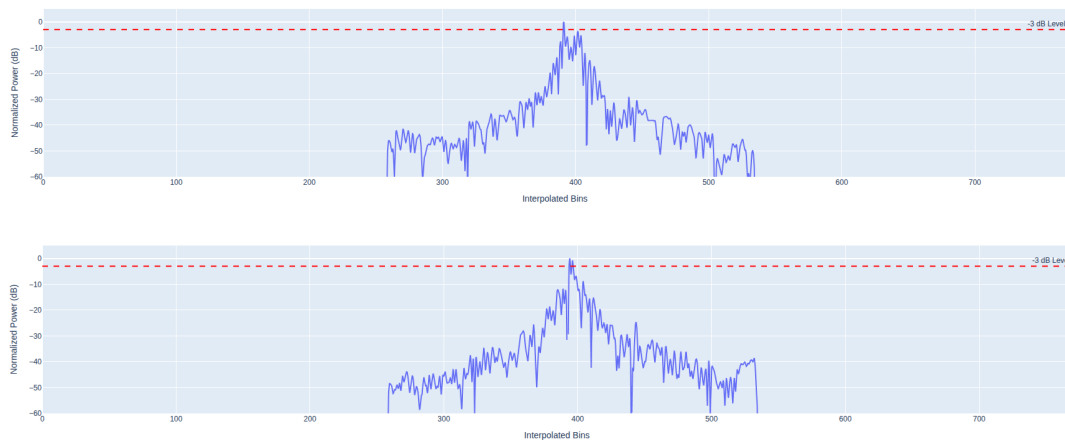
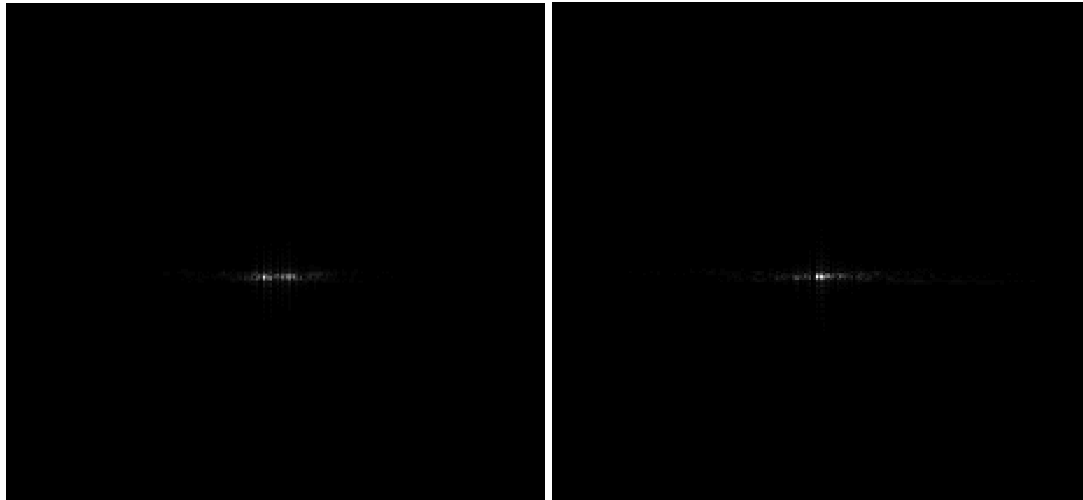
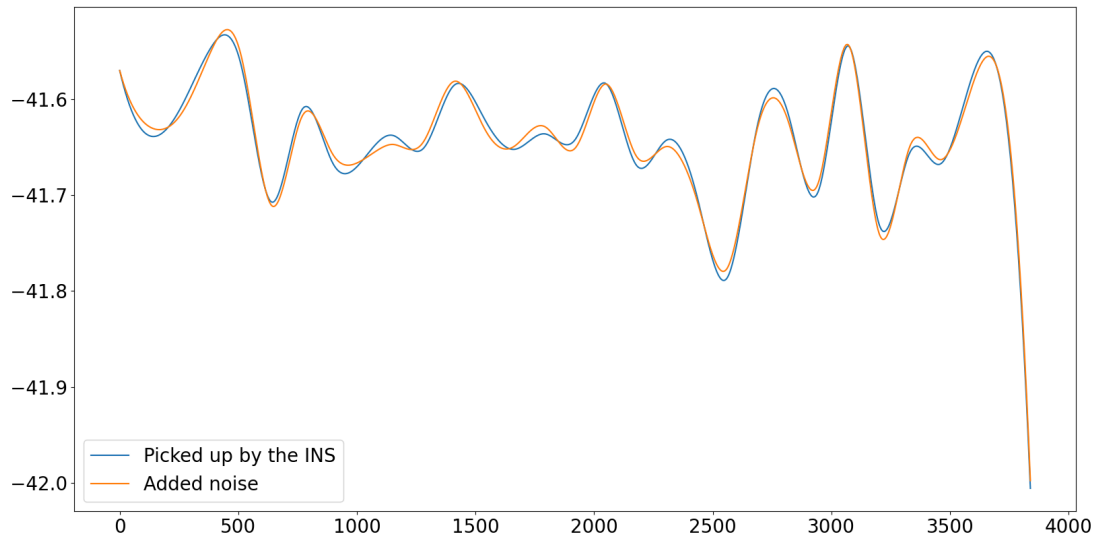


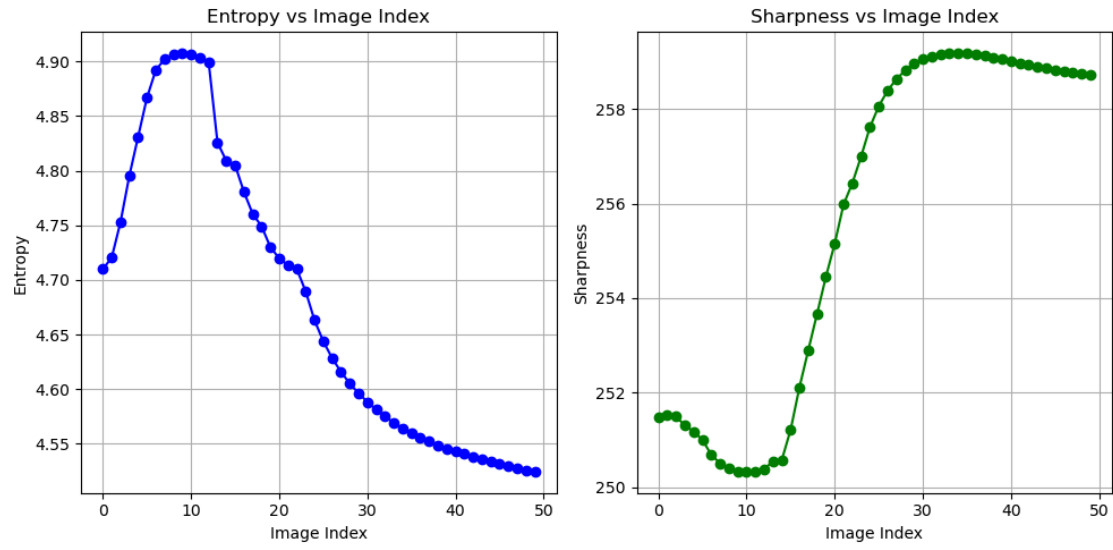


Low-frequency High-Amplitude sinusoidal noise

```
noise = 0.01 * np.sin(2*np.pi*0.5*time)
```

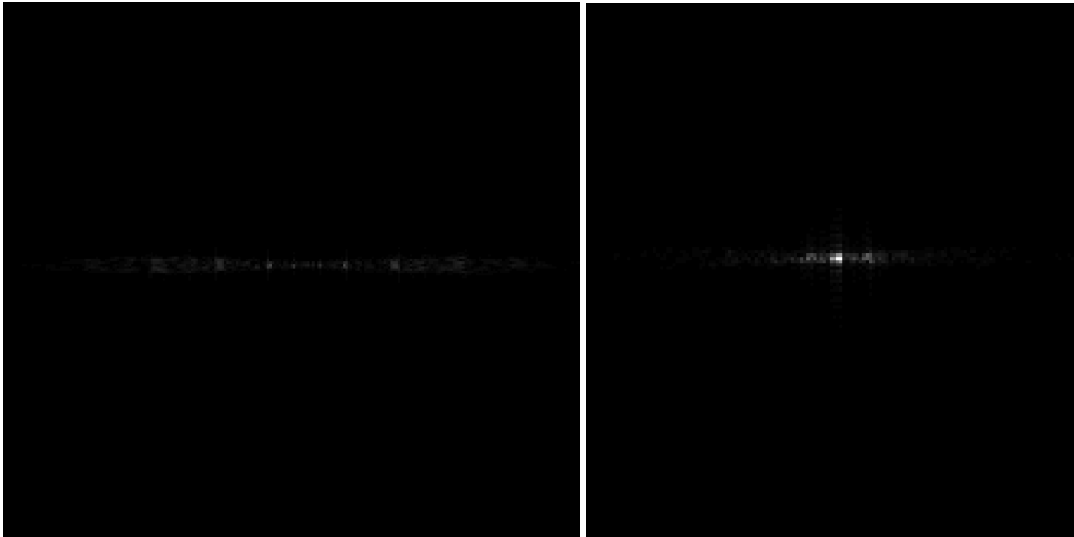
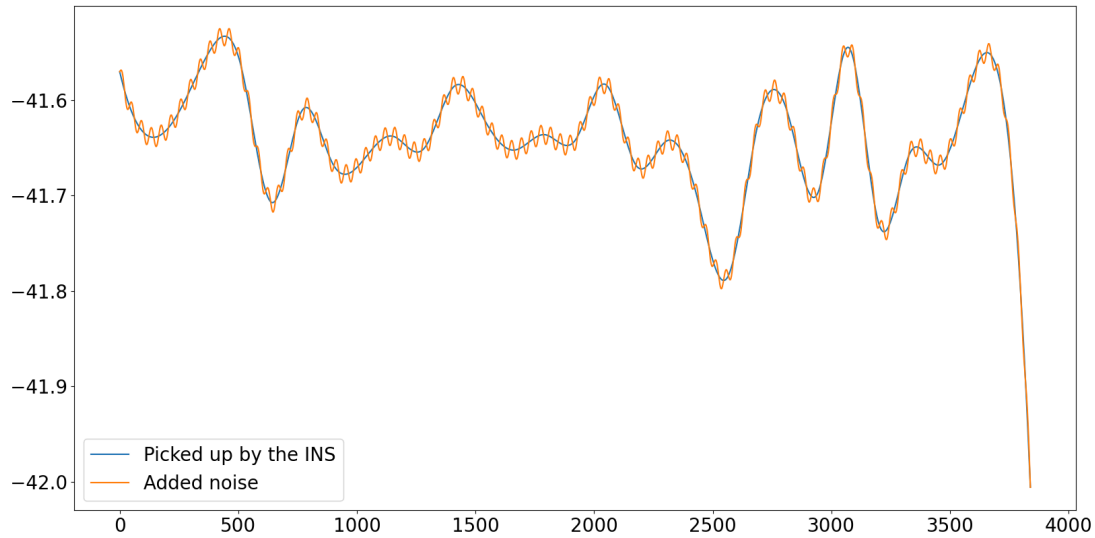
1cm amplitude

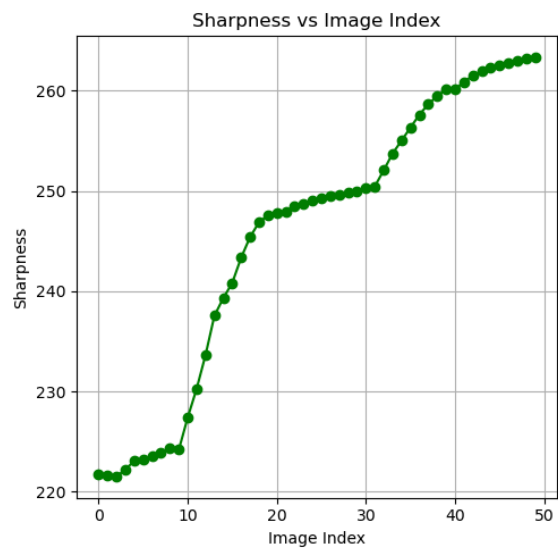
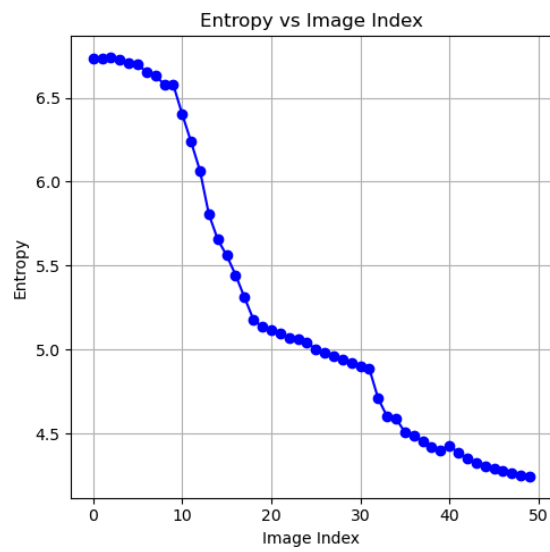
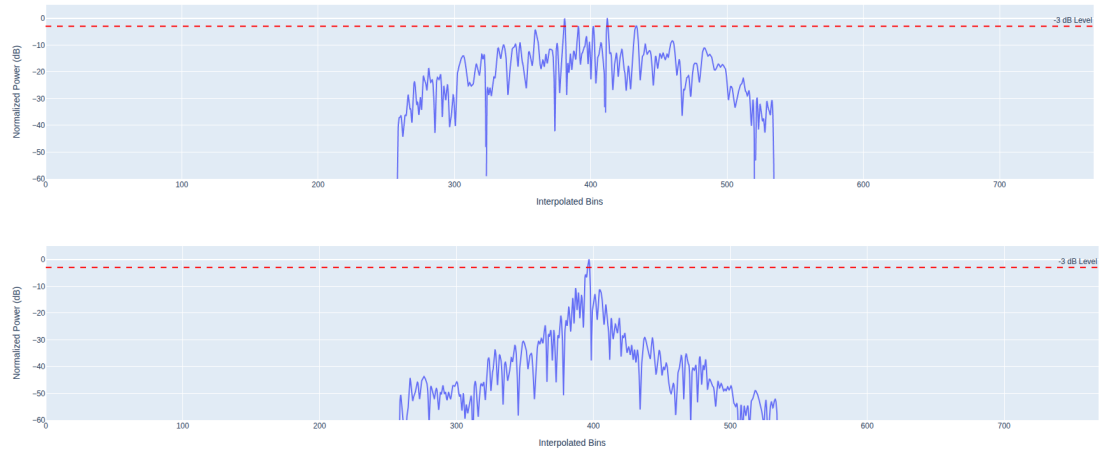




Medium frequency medium amplitude sinusoidal noise

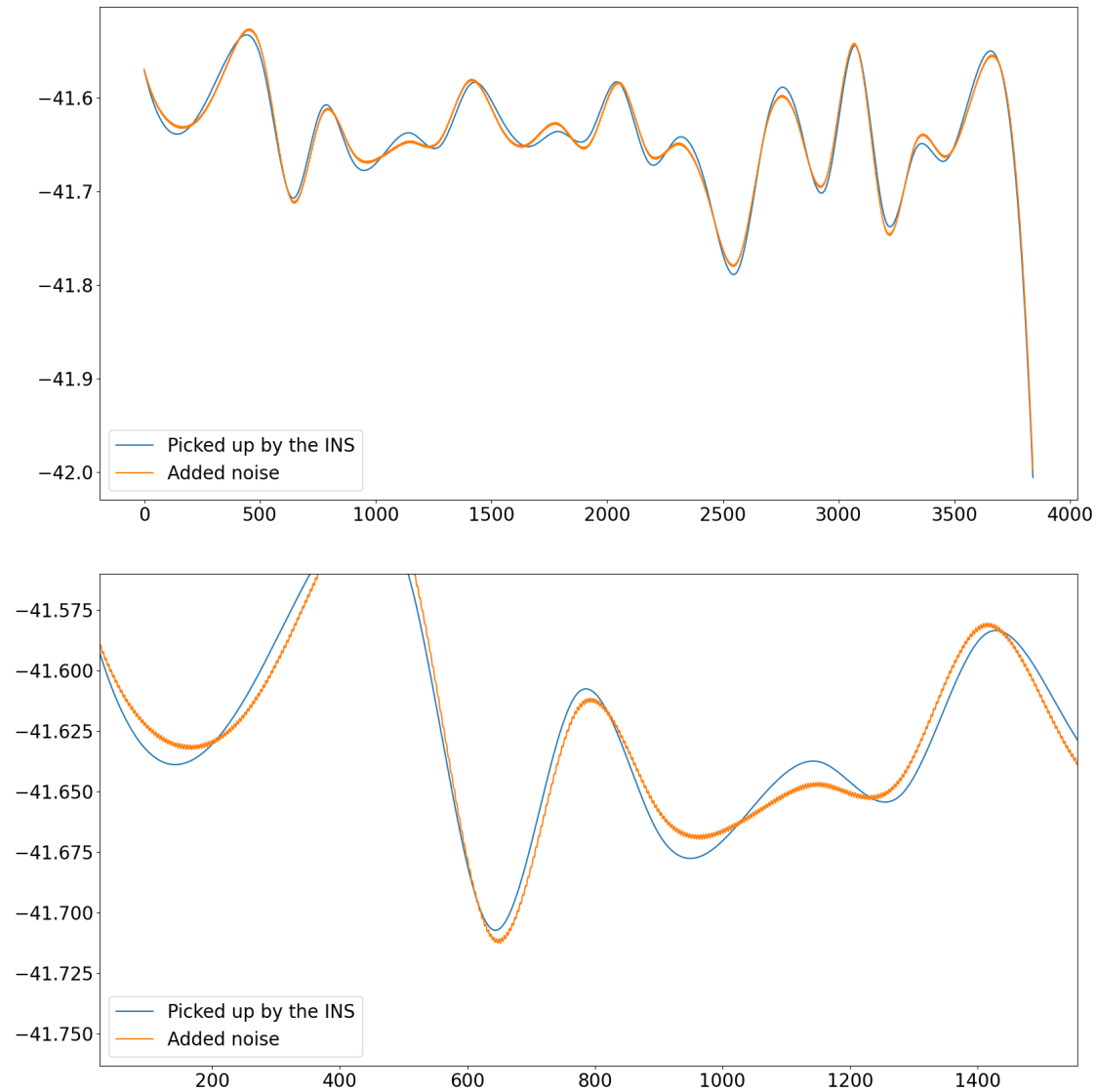
```
noise = 0.01 * np.sin(2*np.pi*5*time)
```





High frequency + Low Frequency Sin noise (reproc)

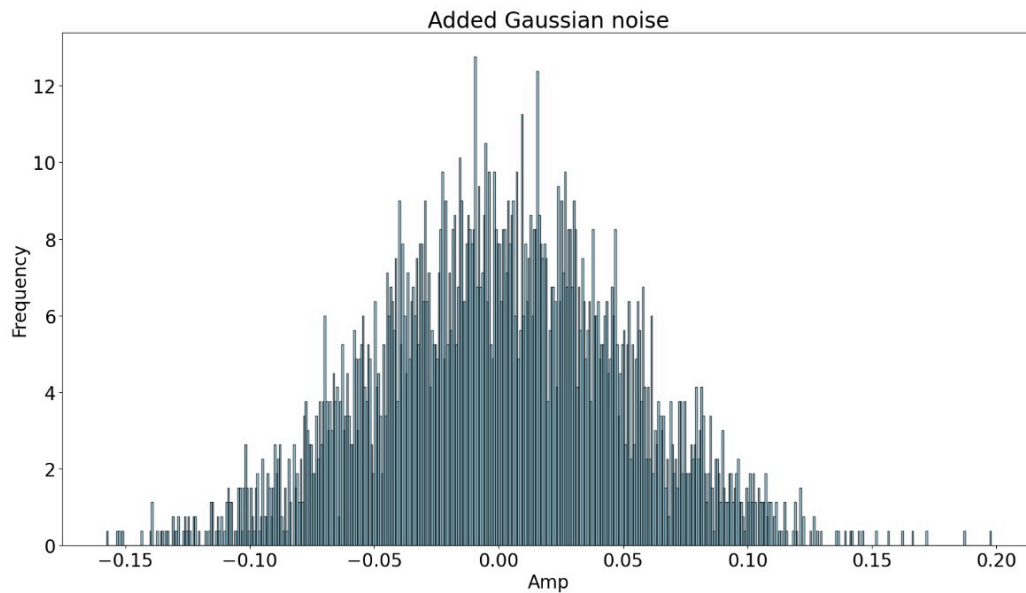
```
noise = 0.01 * np.sin(2*np.pi*0.5*time) + 0.001 * np.sin(2* np.pi * 50 *  
time)
```



Gaussian noise without filtering

```
noise = 0.01 * np.random.normal(loc=1e-5, scale=5, size=len(n))
```

loc = mean; scale = standard deviation;



Added noise histogram

The initial image was bad, but subsequent iterations did not make it better.

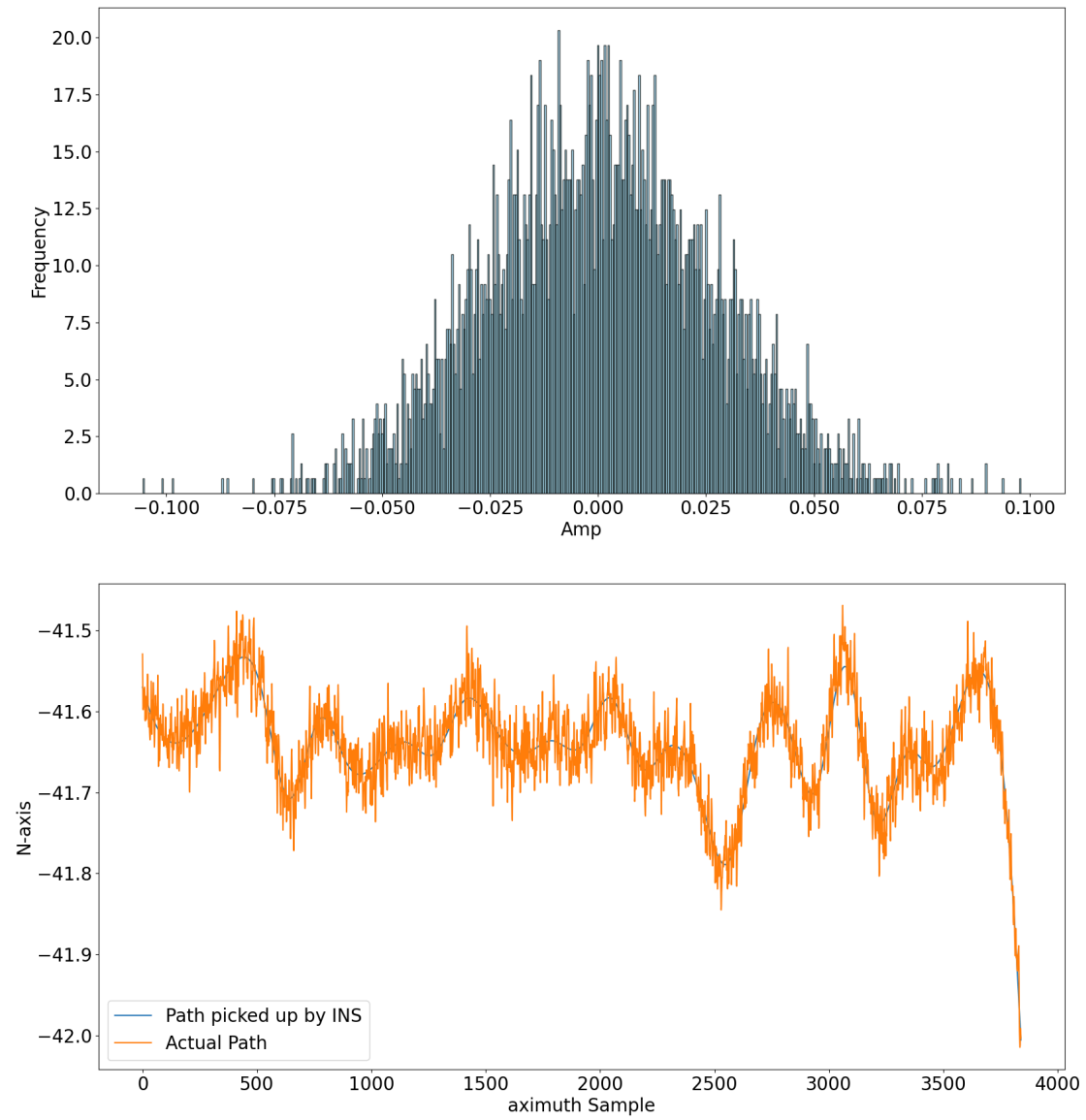
Lowpass filtering through Gaussian filter (noise smoothening)

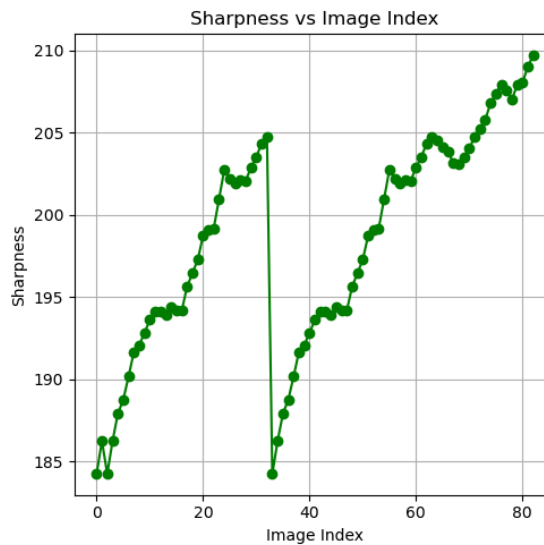
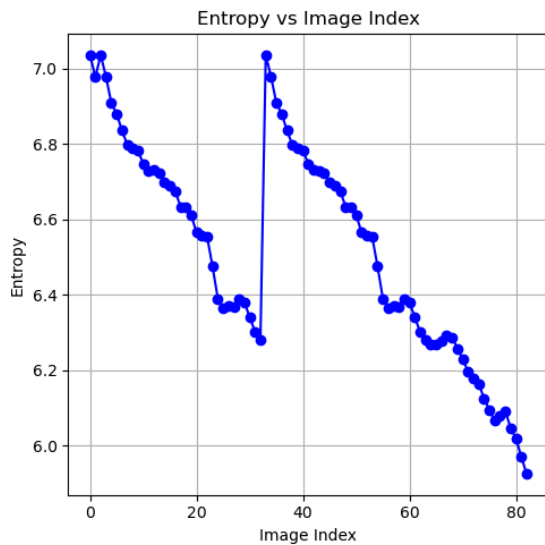
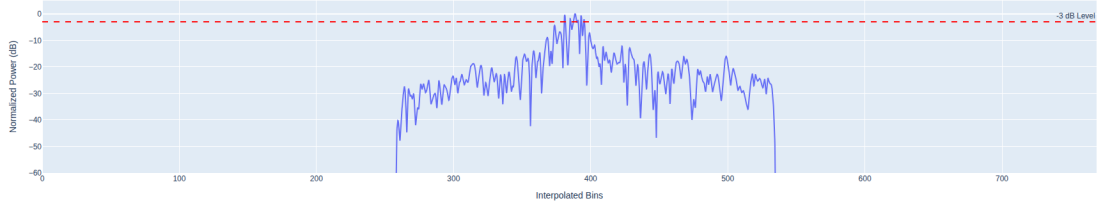
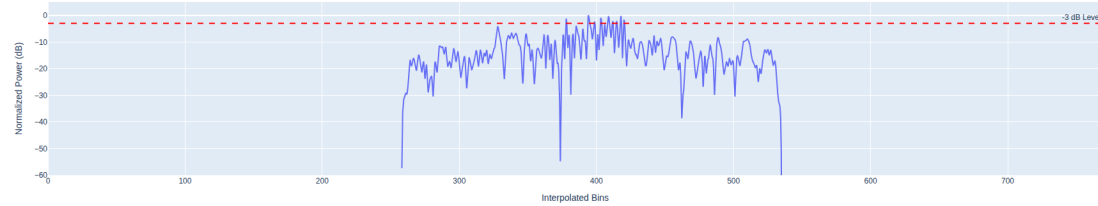
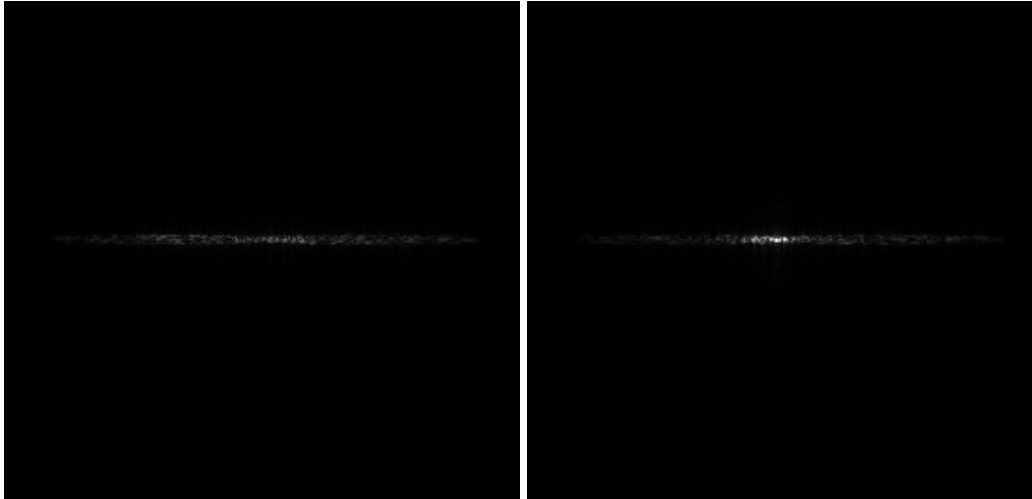
Define the **cutoff frequency** f_c as the frequency where the gain drops to a certain level (commonly -3 dB, or ~ 0.707 of max gain).

$$e^{-2\pi^2 \sigma^2 f_c^2} = \ln\left(\frac{1}{\sqrt{2}}\right)$$

$$f_c \approx \frac{\sqrt{\ln 2}}{2\pi\sigma} = \frac{0.4697 * f_s}{\sigma} = \frac{1803.648}{\sigma}$$

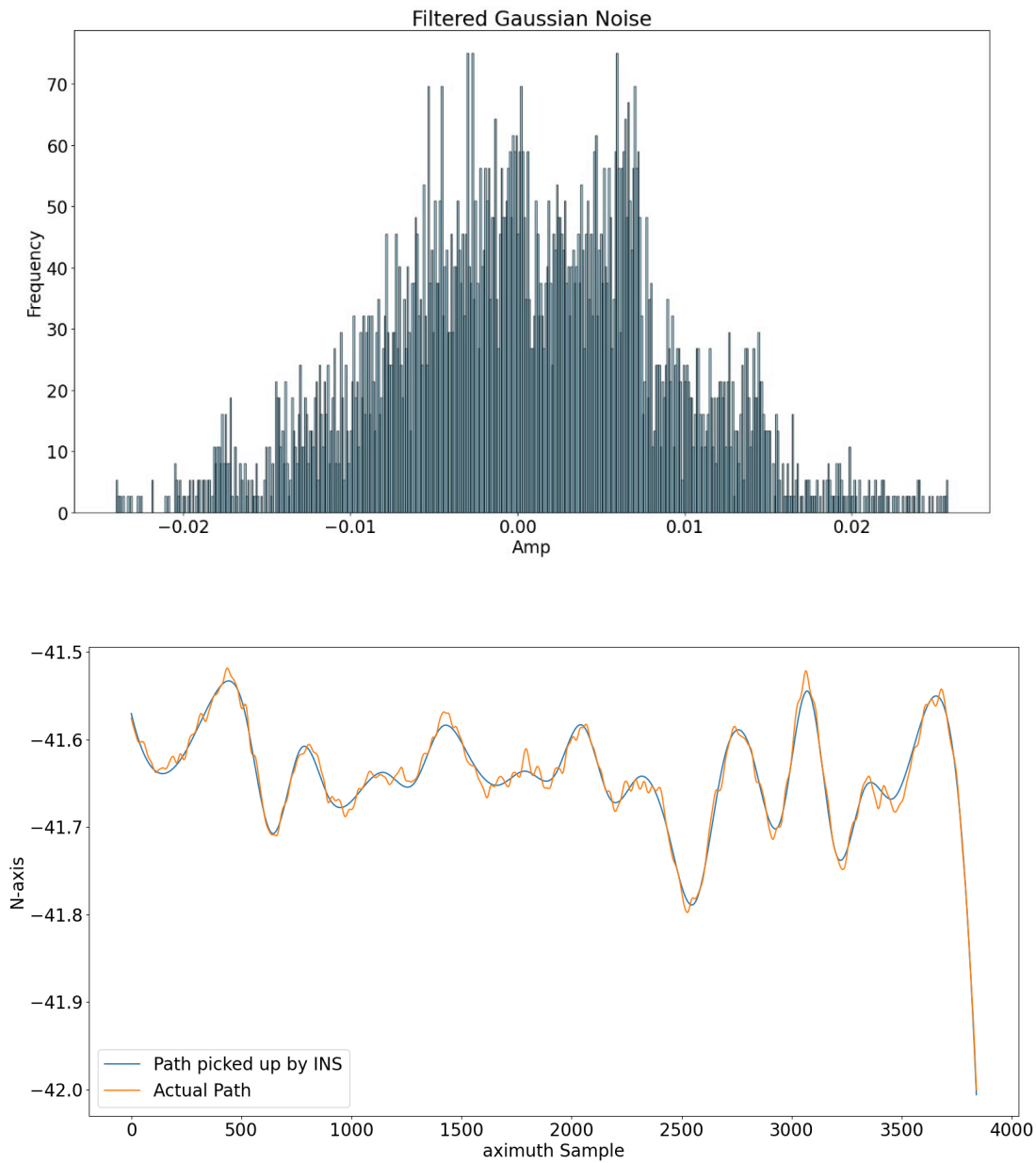
Unfiltered Gaussian noise

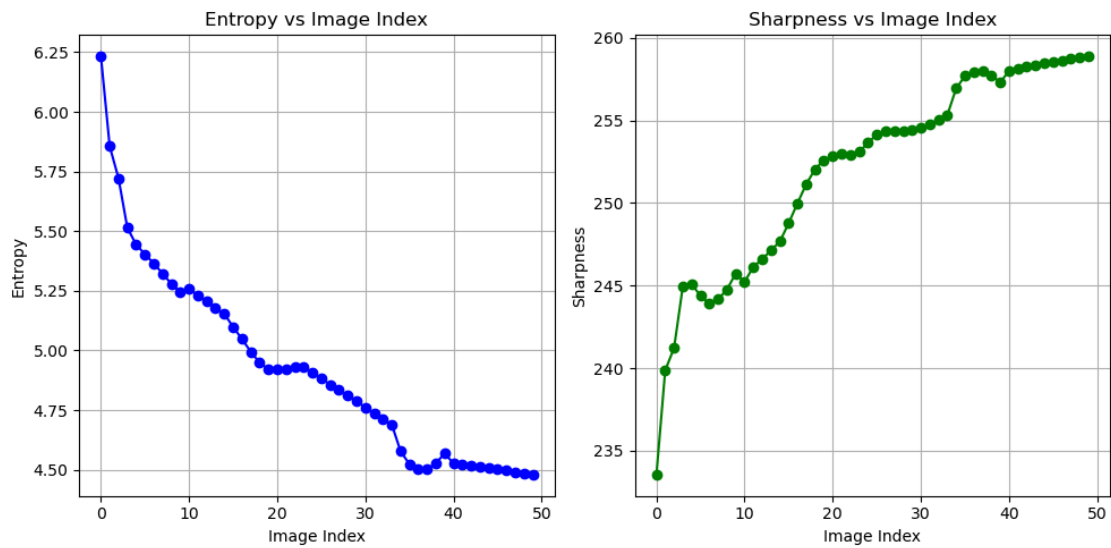
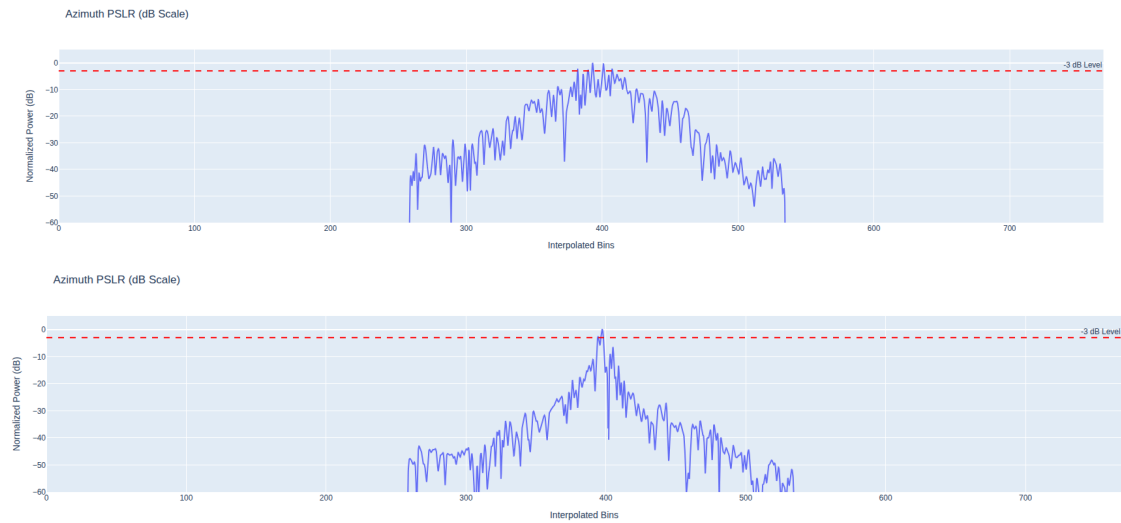
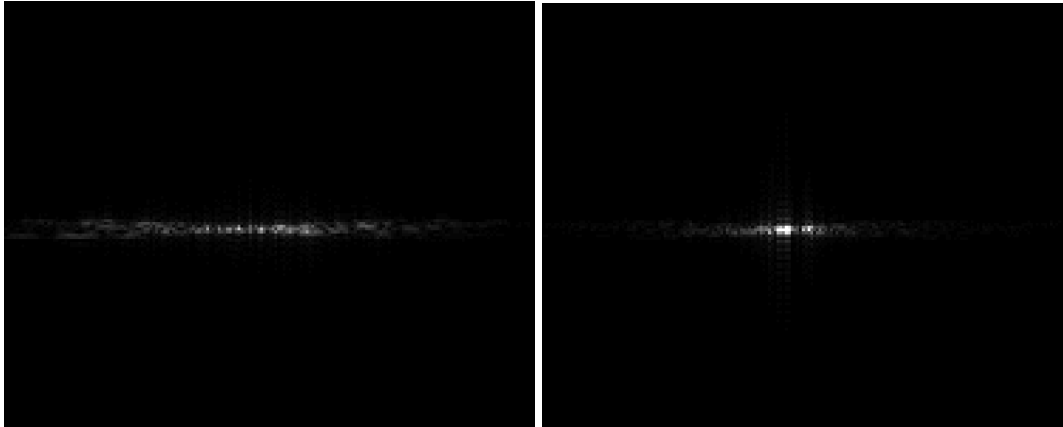




[*] Please note that the entropy and sharpness per iteration needs to be considered from image index 30. For some unknown reason the tool would redo all calculations from 30th index

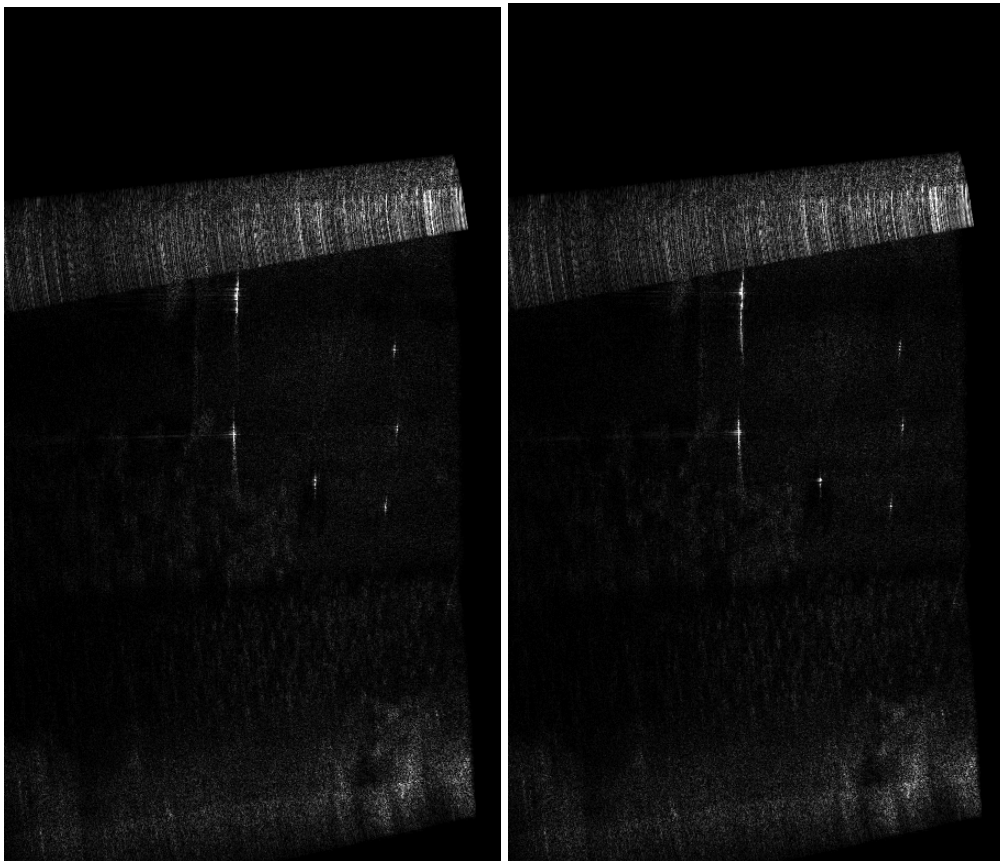
Filtered Gaussian noise (lowpass sigma = 10)



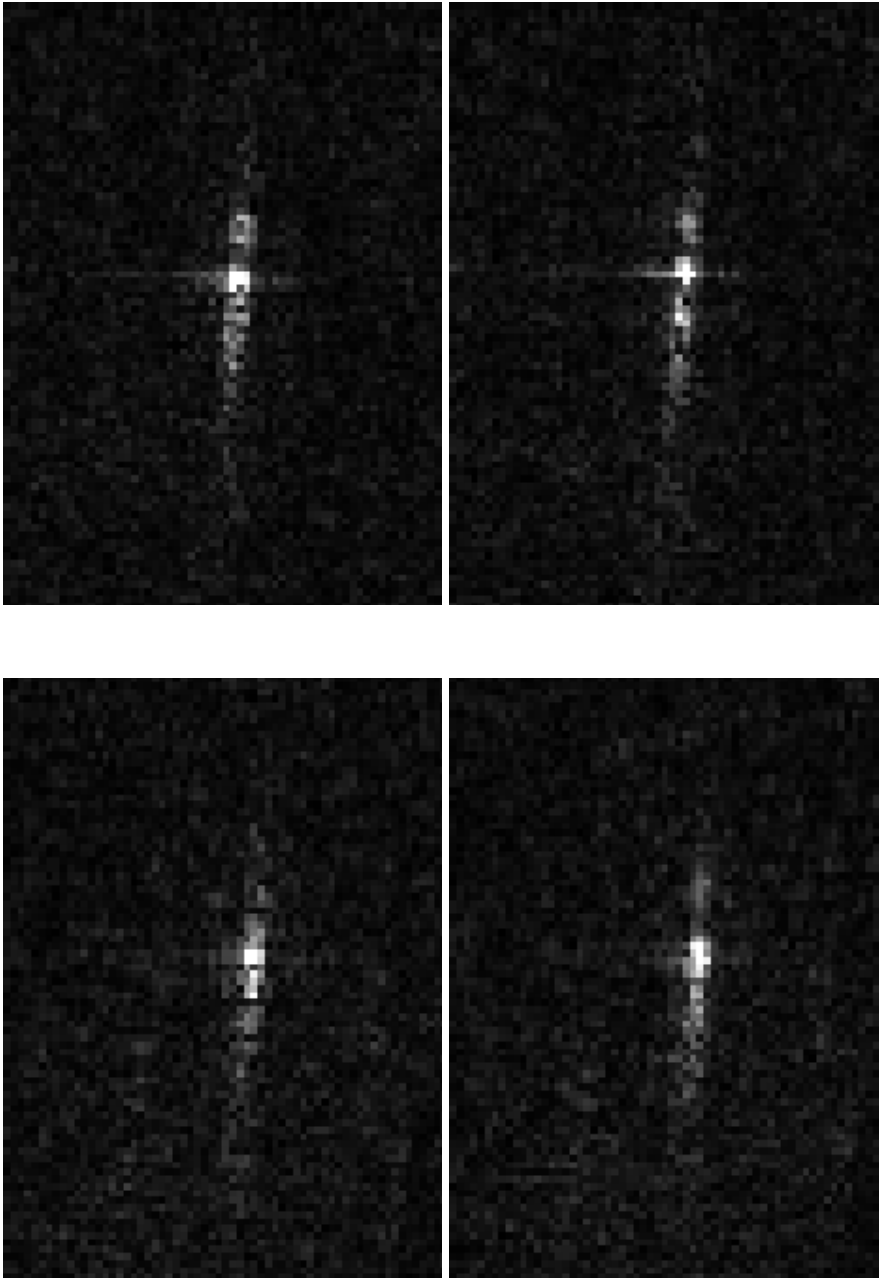


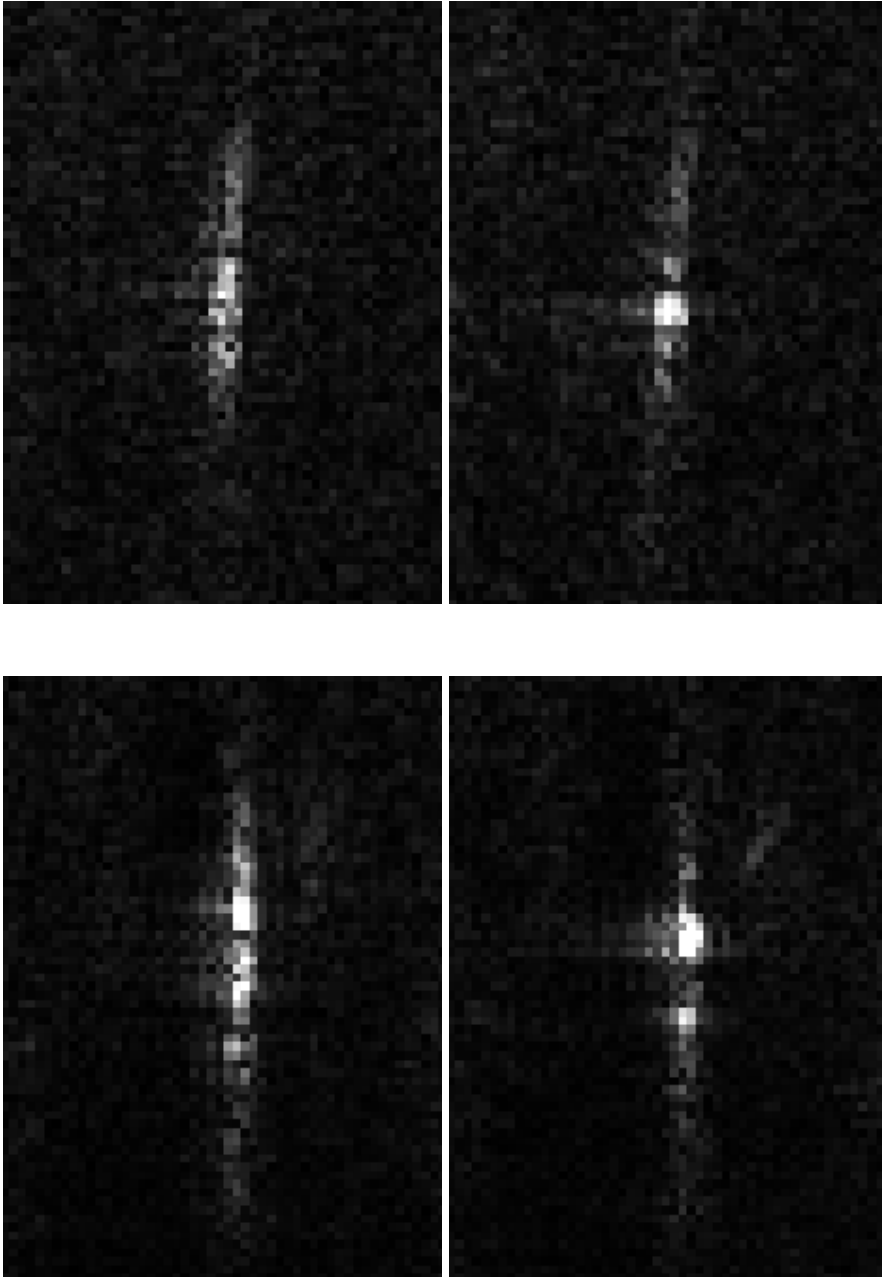
Sensor-1 IdeaForge Platform Test

Overall Comparison

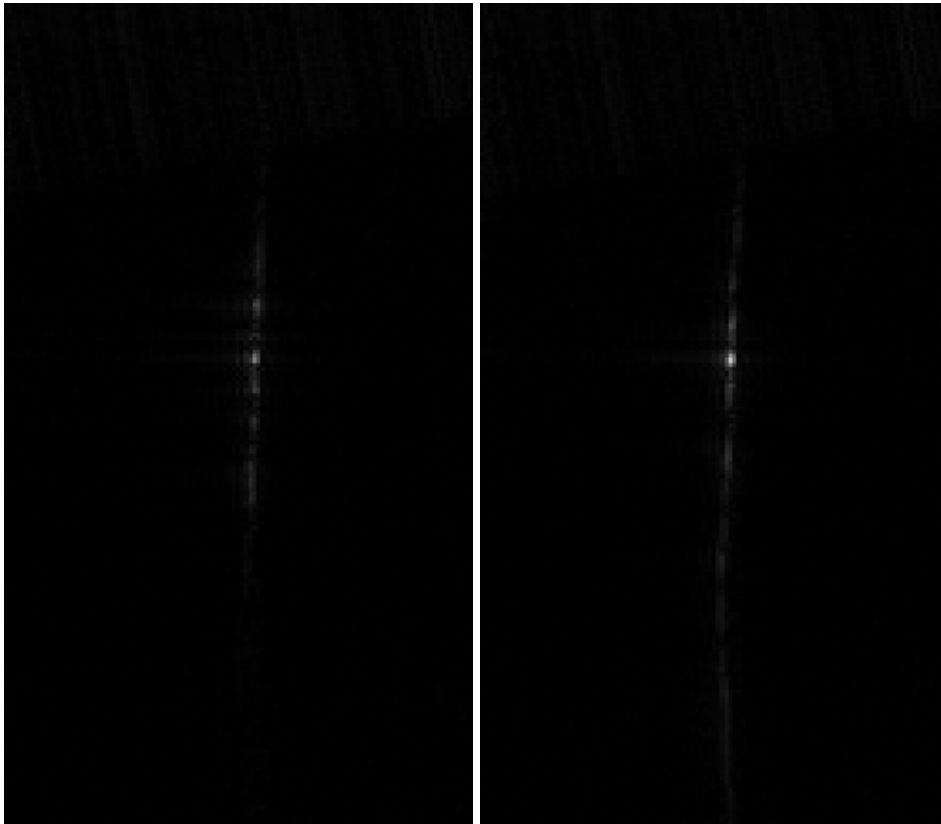
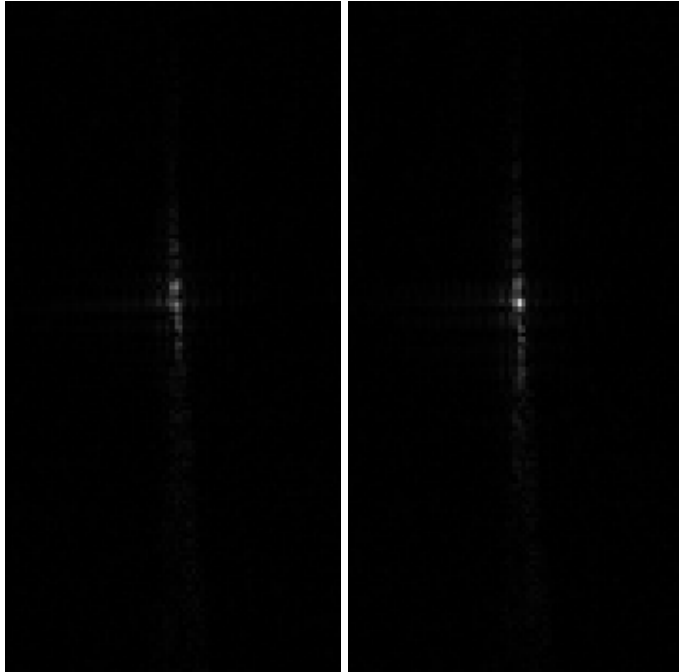


Corner Reflector Level comparison

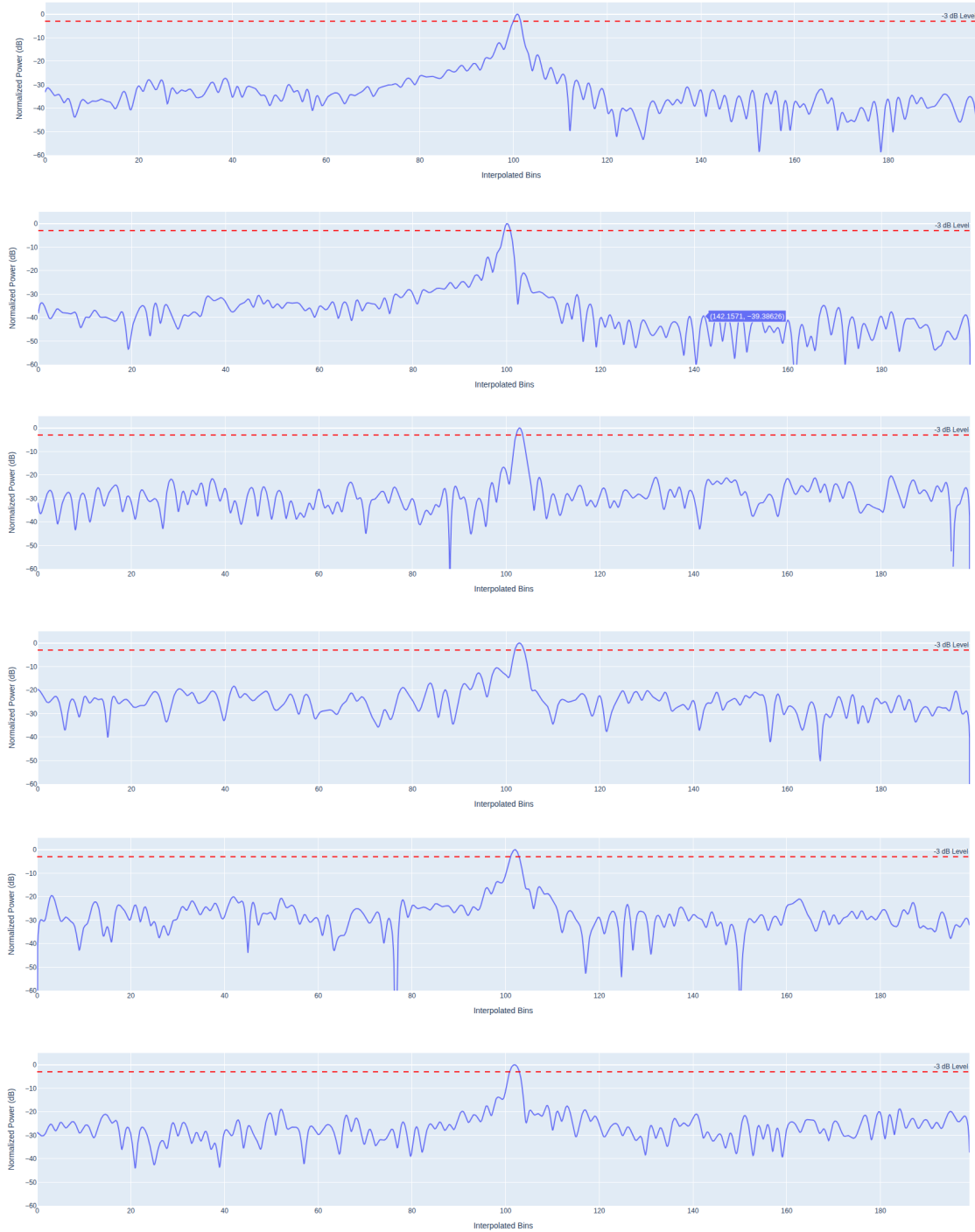




Changed the histogram level for better qualitative comparison



Iteration 0
5 6 4 1 2 3



Iteration 50

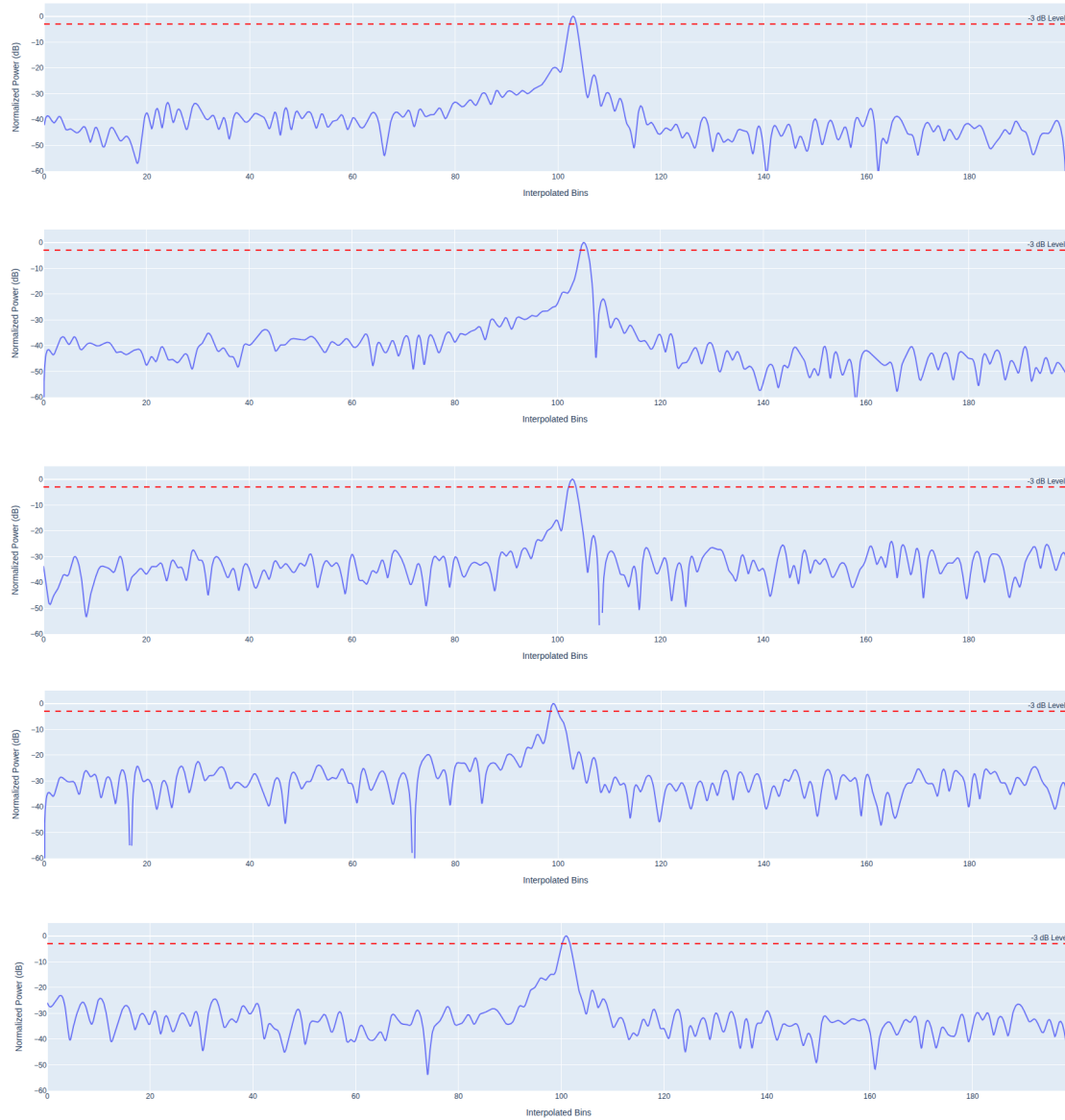
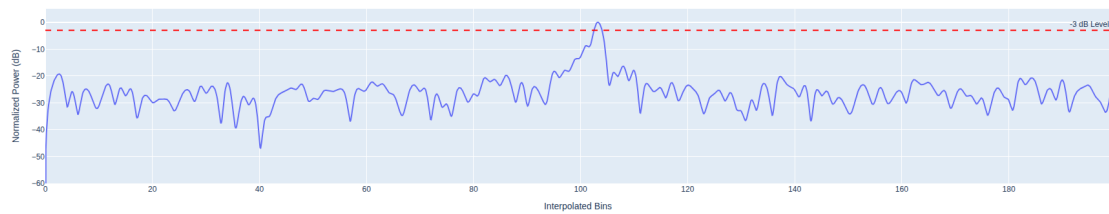
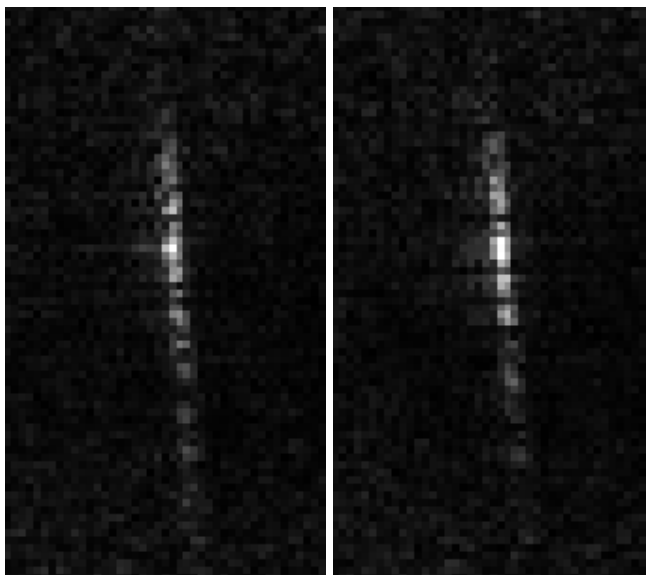
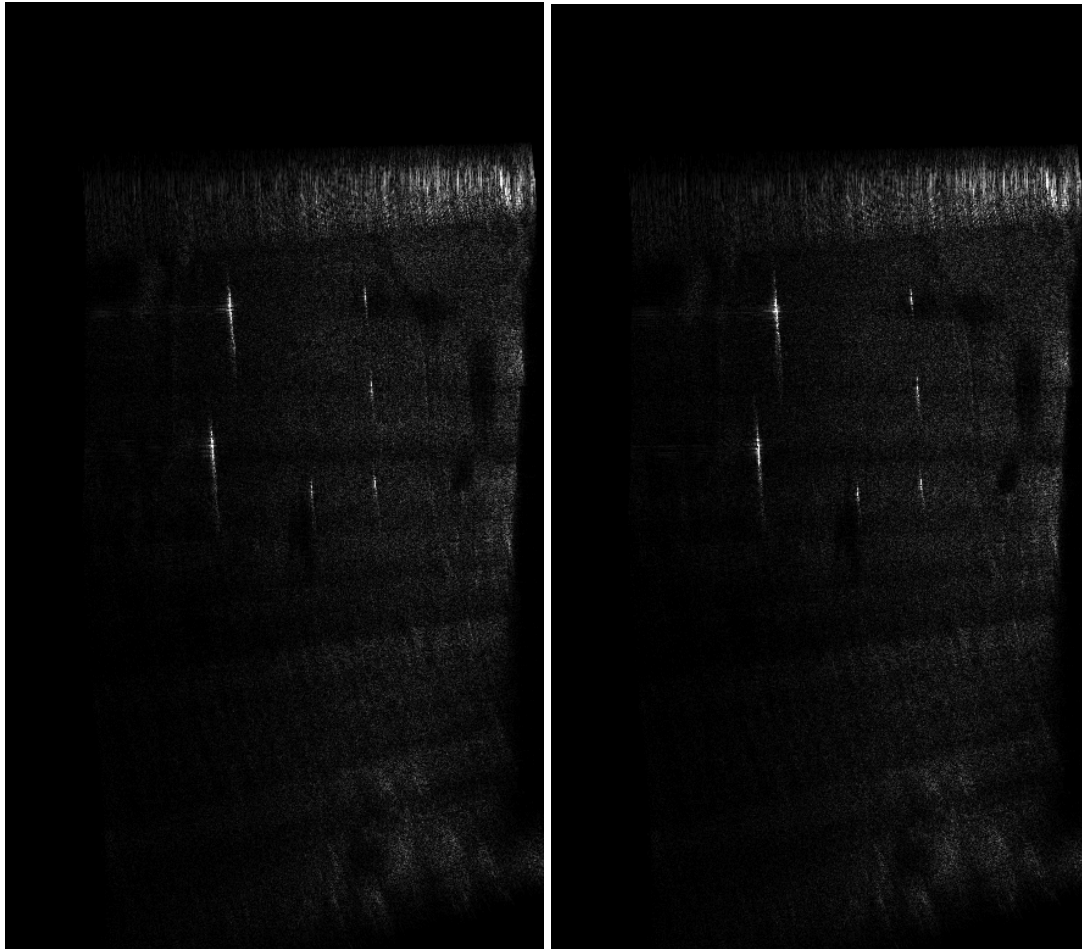
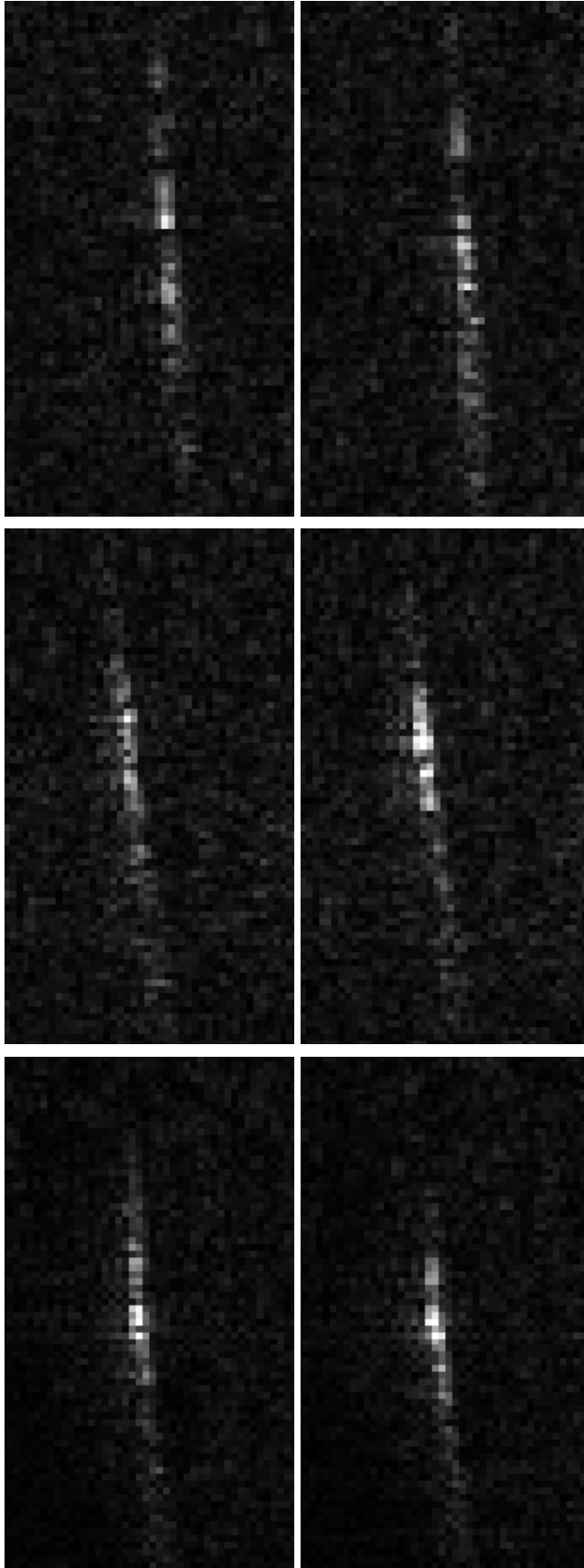


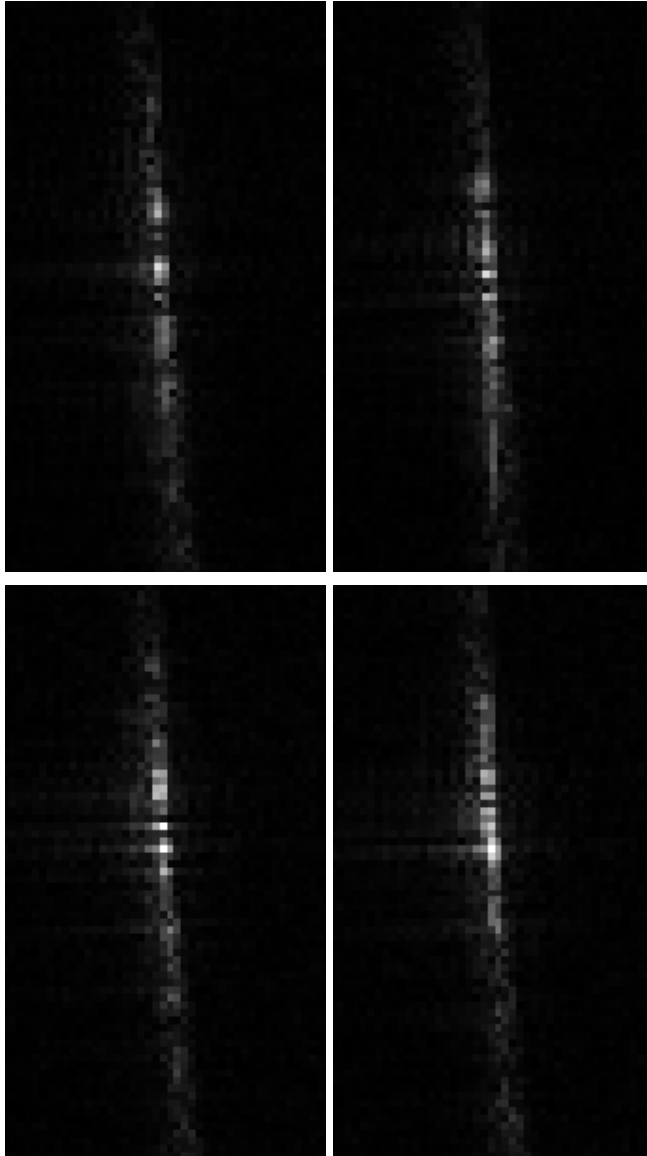
Fig: nsjldvhsdjl

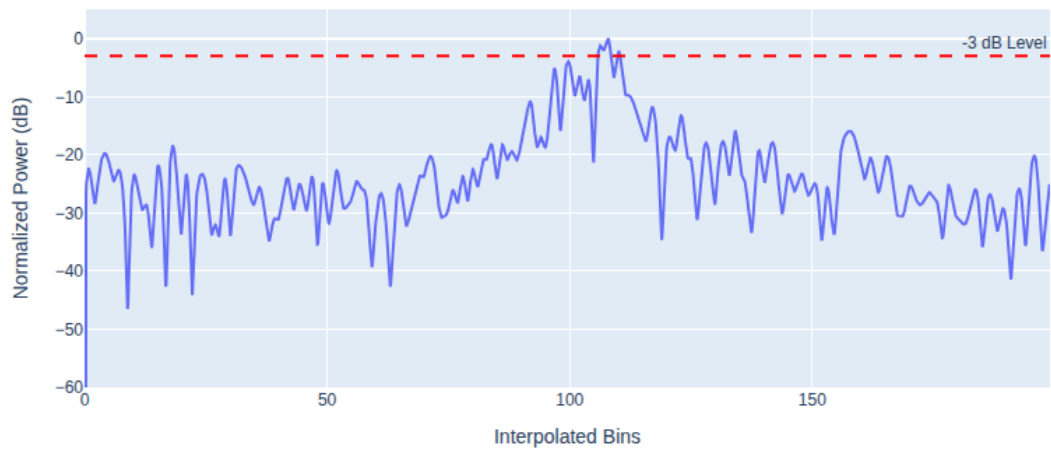
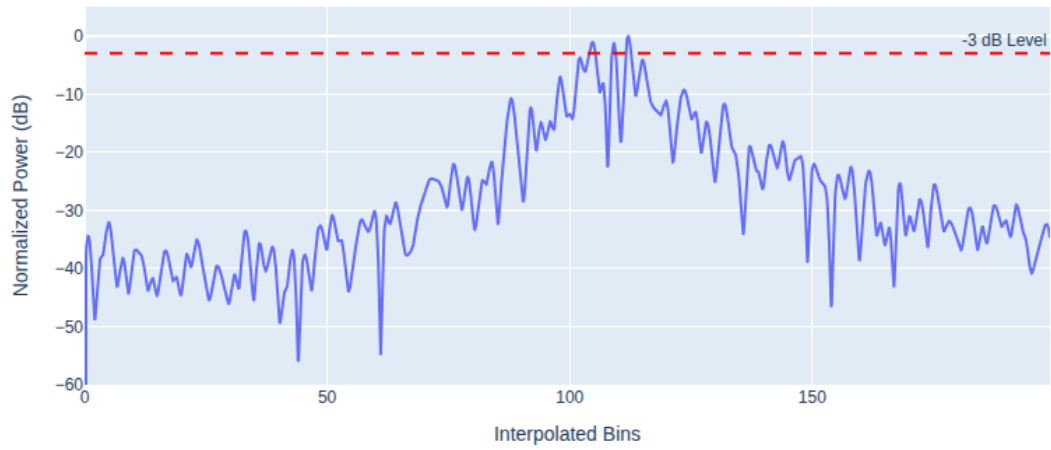
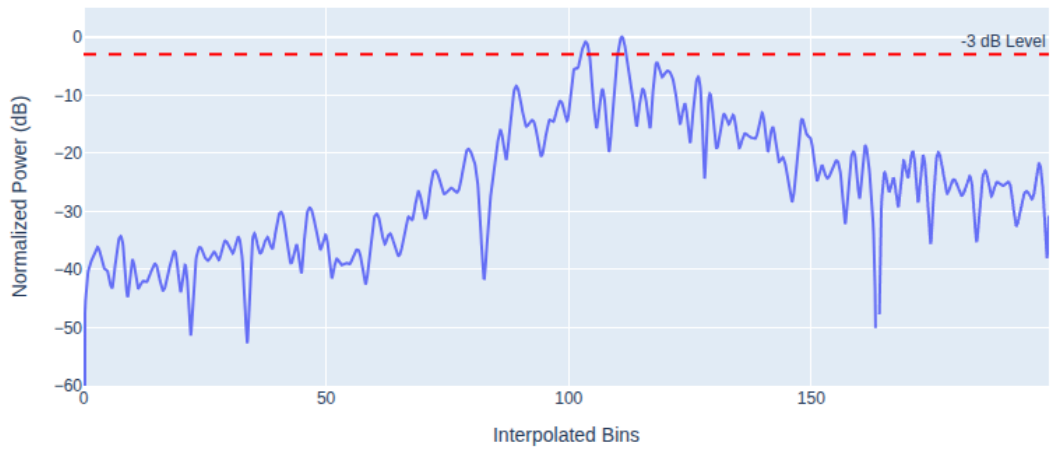


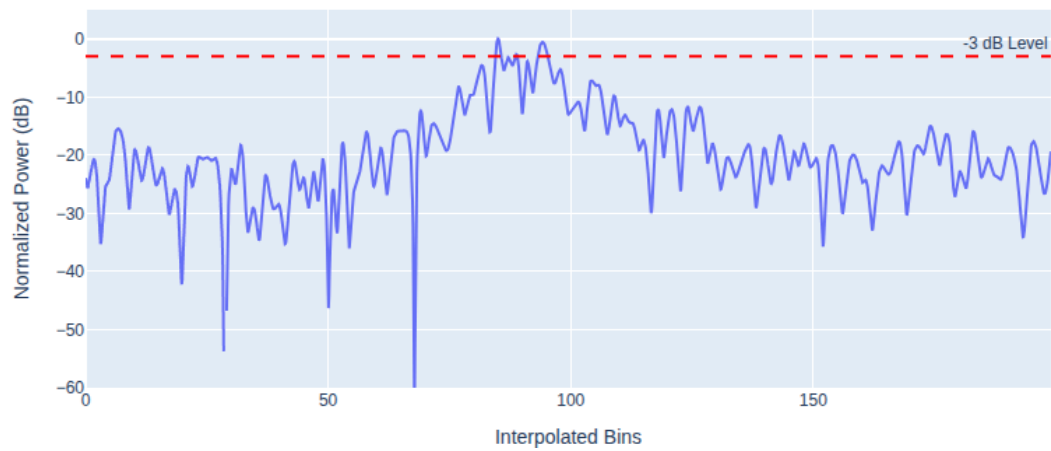
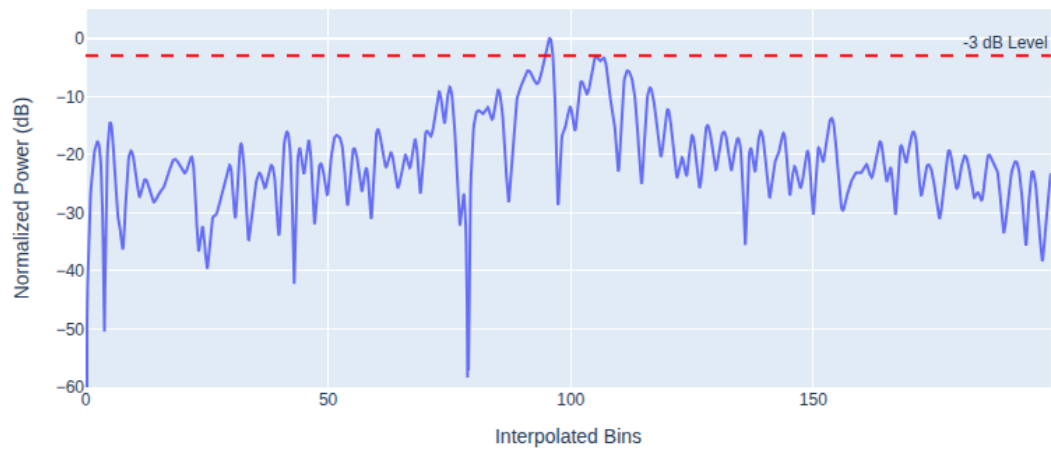
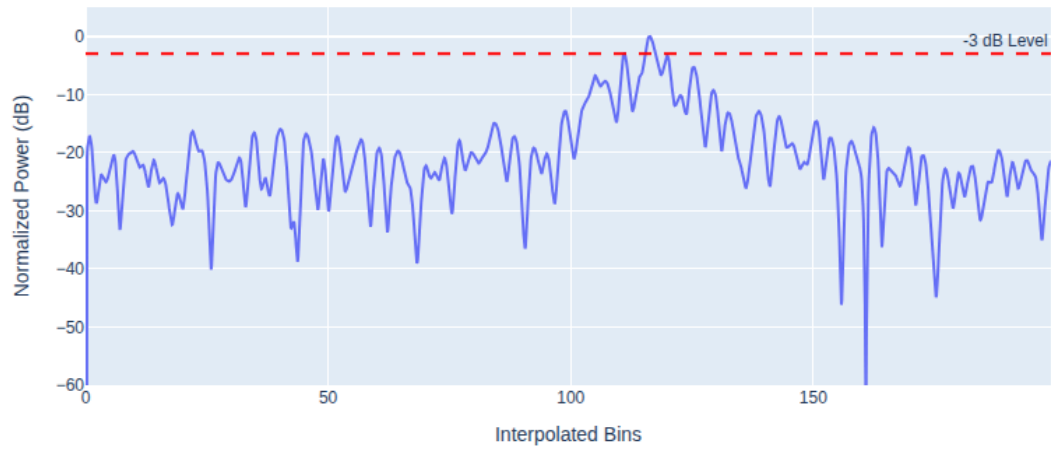
Sensor-1 Scandron platform test

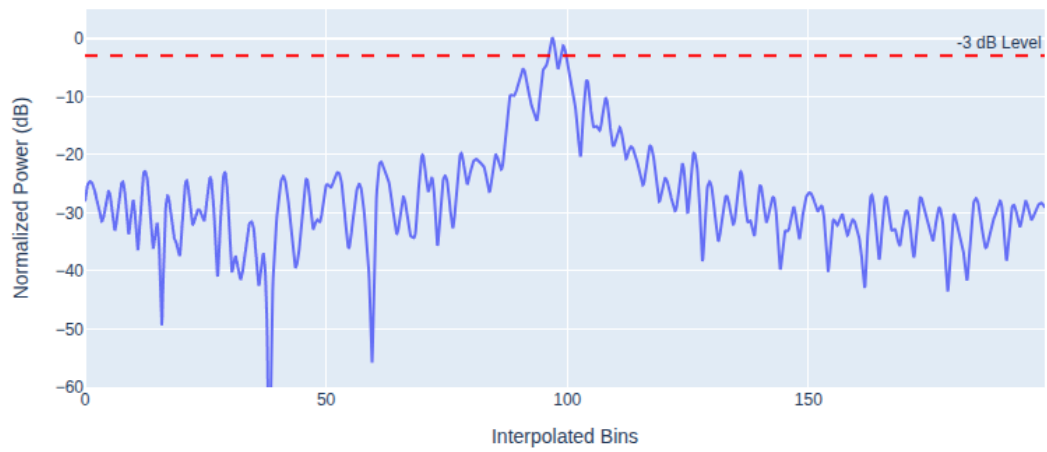
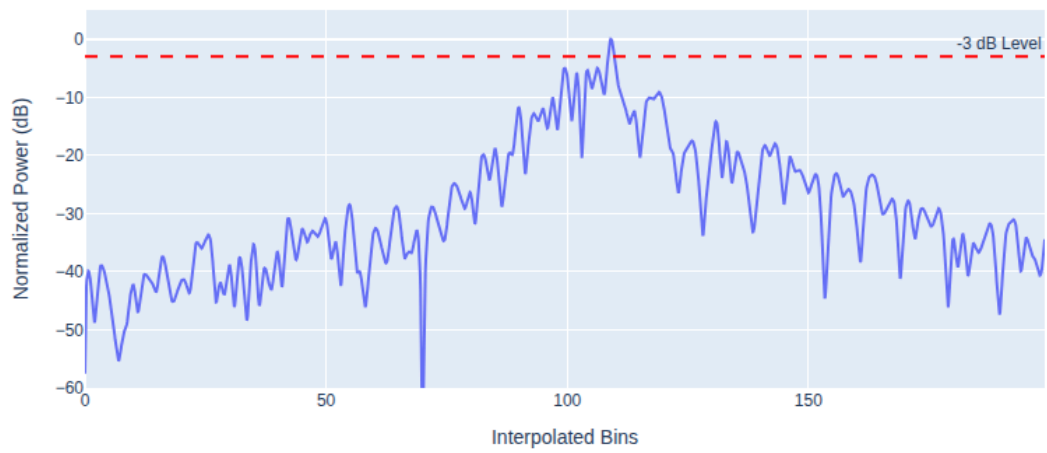
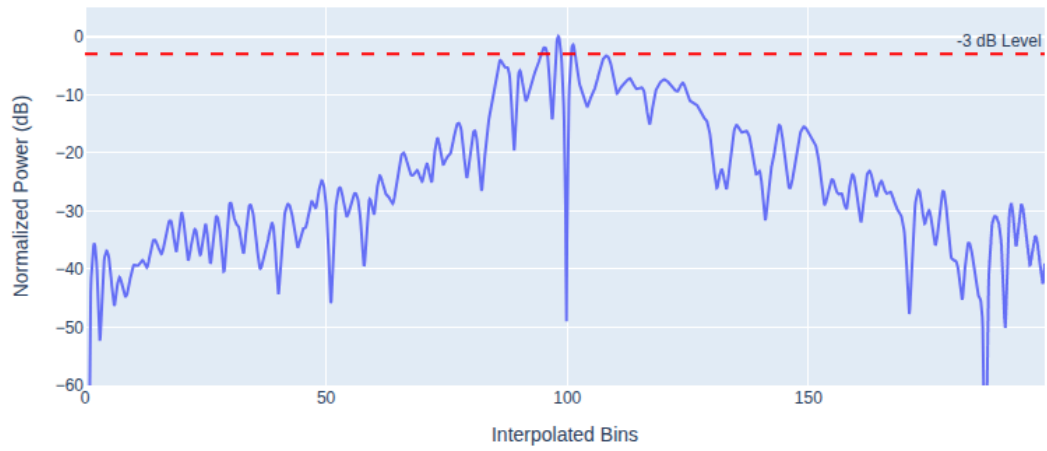


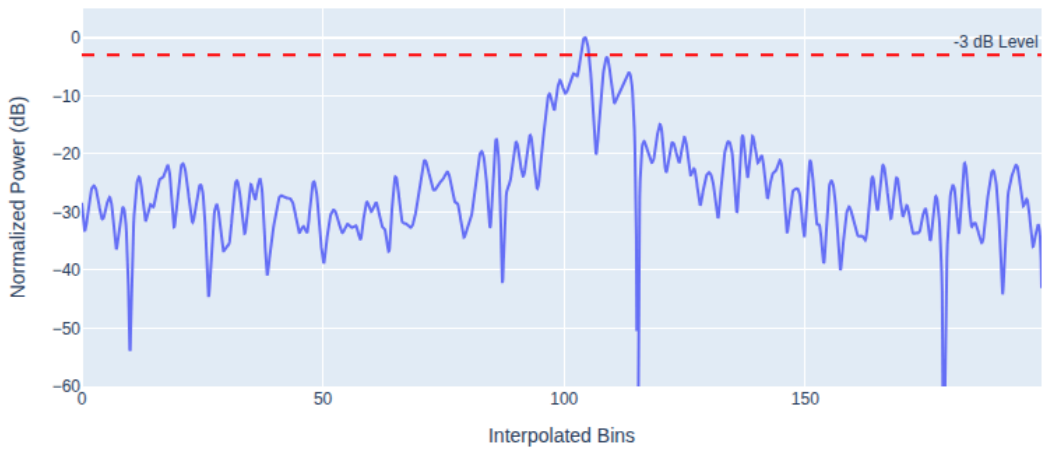
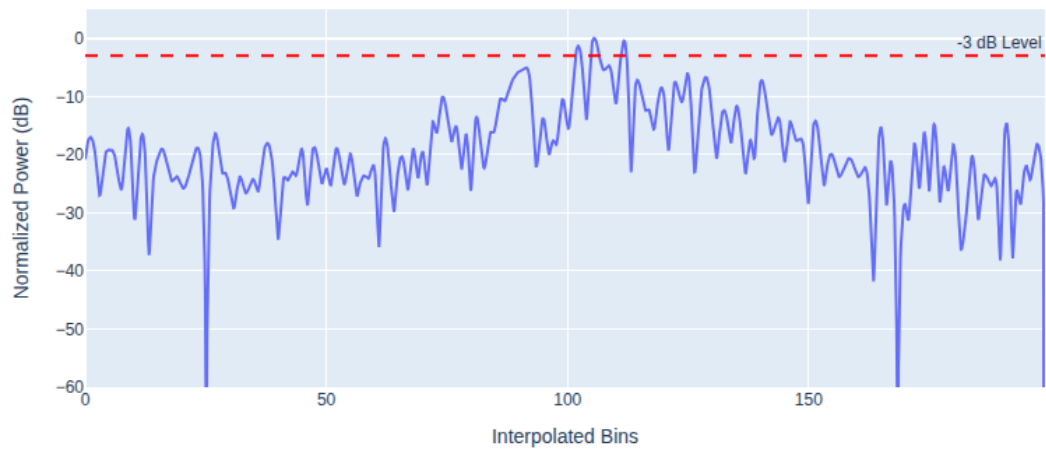
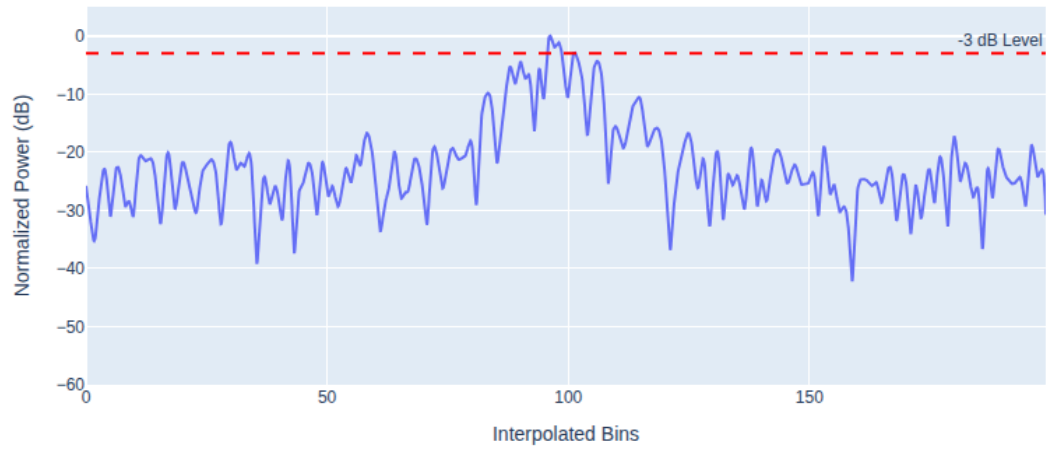












Conclusion